NN Background

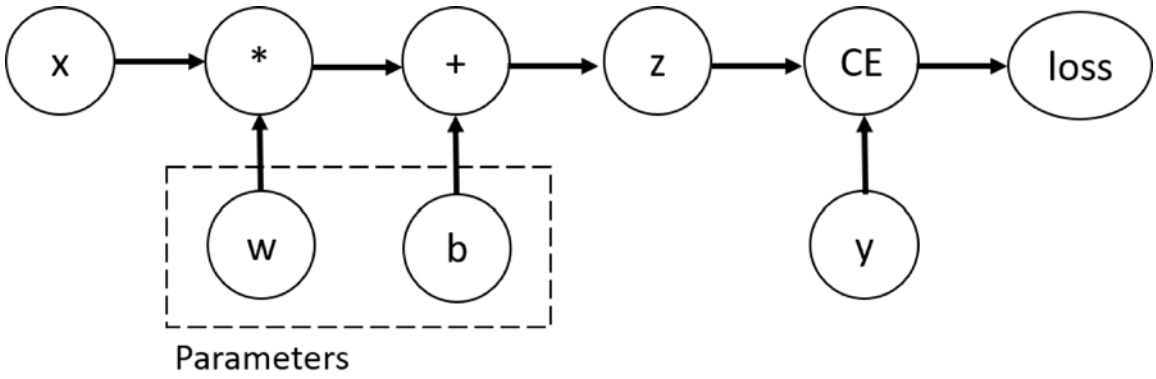
Neural networks (NNs) are a collection of nested functions that are executed on some input data. These functions are defined by *parameters* (consisting of weights and biases), which in PyTorch are stored in tensors.

Training a NN happens in two steps:

Forward Propagation: In forward prop, the NN makes its best guess about the correct output. It runs the input data through each of its functions to make this guess.

Backward Propagation: In backprop, the NN adjusts its parameters proportionate to the error in its guess. It does this by traversing backwards from the output, collecting the derivatives of the error with respect to the parameters of the functions (*gradients*), and optimizing the parameters using gradient descent.

For a more detailed walkthrough of backprop, check out this [video from 3Blue1Brown] (https://www.youtube.com/watch?v=tIeHLnjs5U8).



Using PyTorch

Let's take a look at a single training step. For this example, we load a pretrained resnet18 model from `torchvision`. We create a random data tensor to represent a single image with 3 channels, and height & width of 64, and its corresponding `label` initialized to some random values.

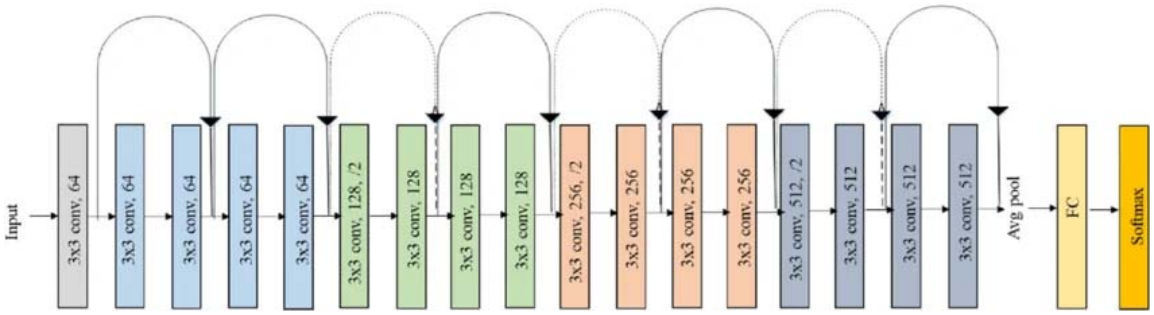
`torchvision` **Key Components:**

- **Datasets**: Provides standard computer vision loaders like ImageNet, CIFAR-10, and MNIST.
- **Models**: Includes definitions for classic and modern architectures (like ResNet, VGG, and MobileNet) with pre-trained weights.

- **Transforms**: Offers common preprocessing and data augmentation operations for images, videos, and bounding boxes.

In [1]:
```
import torch, torchvision
from torch import nn
```

In [3]:
```
from torchvision.models import resnet18, ResNet18_Weights
```



Download the model and save the model into path to be used later without downloading

The code snippet `model = resnet18(weights=ResNet18_Weights.DEFAULT)` initializes a ResNet-18 model pre-loaded with the highest-performing weights available in the PyTorch library.

By using `ResNet18_Weights.DEFAULT`, PyTorch automatically downloads and applies weights trained on the **ImageNet-1K** dataset (1000 class).

This configures your network for transfer learning or fine-tuning (seen later), saving you massive amounts of computational time and data requirements compared to training from scratch.

In [4]:
```
model = resnet18(weights=ResNet18_Weights.DEFAULT)
print(model)

PATH = r'D:\Teaching\UL\AI\Lab\resnetModel\resnet18.pth'
#save model parameters in dictionary
torch.save(model.state_dict(), PATH) #state_dict is simply a Python dictionary obje
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=Fal
se)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=Fal
se)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bi
as=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
```

```
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
```

```
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=1000, bias=True)
)
```

Notice the "out_features=1000"

Inspecting Model Weights or Tensors

In the `.pt` file contains a model's state dictionary (weights and biases), you can loop through the dictionary keys and inspect individual tensor shapesIf your `.pt` file contains a model's state dictionary (weights and biases), you can loop through the dictionary keys and inspect individual tensor shapes`.

In [5]:
```
import torch

state_dict = torch.load(PATH, map_location="cpu")

for key, value in state_dict.items():
  print(f"{key}: {value.shape}")
```

```
conv1.weight: torch.Size([64, 3, 7, 7])
bn1.weight: torch.Size([64])
bn1.bias: torch.Size([64])
bn1.running_mean: torch.Size([64])
bn1.running_var: torch.Size([64])
bn1.num_batches_tracked: torch.Size([])
layer1.0.conv1.weight: torch.Size([64, 64, 3, 3])
layer1.0.bn1.weight: torch.Size([64])
layer1.0.bn1.bias: torch.Size([64])
layer1.0.bn1.running_mean: torch.Size([64])
layer1.0.bn1.running_var: torch.Size([64])
layer1.0.bn1.num_batches_tracked: torch.Size([])
layer1.0.conv2.weight: torch.Size([64, 64, 3, 3])
layer1.0.bn2.weight: torch.Size([64])
layer1.0.bn2.bias: torch.Size([64])
layer1.0.bn2.running_mean: torch.Size([64])
layer1.0.bn2.running_var: torch.Size([64])
layer1.0.bn2.num_batches_tracked: torch.Size([])
layer1.1.conv1.weight: torch.Size([64, 64, 3, 3])
layer1.1.bn1.weight: torch.Size([64])
layer1.1.bn1.bias: torch.Size([64])
layer1.1.bn1.running_mean: torch.Size([64])
layer1.1.bn1.running_var: torch.Size([64])
layer1.1.bn1.num_batches_tracked: torch.Size([])
layer1.1.conv2.weight: torch.Size([64, 64, 3, 3])
layer1.1.bn2.weight: torch.Size([64])
layer1.1.bn2.bias: torch.Size([64])
layer1.1.bn2.running_mean: torch.Size([64])
layer1.1.bn2.running_var: torch.Size([64])
layer1.1.bn2.num_batches_tracked: torch.Size([])
layer2.0.conv1.weight: torch.Size([128, 64, 3, 3])
layer2.0.bn1.weight: torch.Size([128])
layer2.0.bn1.bias: torch.Size([128])
layer2.0.bn1.running_mean: torch.Size([128])
layer2.0.bn1.running_var: torch.Size([128])
layer2.0.bn1.num_batches_tracked: torch.Size([])
layer2.0.conv2.weight: torch.Size([128, 128, 3, 3])
layer2.0.bn2.weight: torch.Size([128])
layer2.0.bn2.bias: torch.Size([128])
layer2.0.bn2.running_mean: torch.Size([128])
layer2.0.bn2.running_var: torch.Size([128])
layer2.0.bn2.num_batches_tracked: torch.Size([])
layer2.0.downsample.0.weight: torch.Size([128, 64, 1, 1])
layer2.0.downsample.1.weight: torch.Size([128])
layer2.0.downsample.1.bias: torch.Size([128])
layer2.0.downsample.1.running_mean: torch.Size([128])
layer2.0.downsample.1.running_var: torch.Size([128])
layer2.0.downsample.1.num_batches_tracked: torch.Size([])
layer2.1.conv1.weight: torch.Size([128, 128, 3, 3])
layer2.1.bn1.weight: torch.Size([128])
layer2.1.bn1.bias: torch.Size([128])
layer2.1.bn1.running_mean: torch.Size([128])
layer2.1.bn1.running_var: torch.Size([128])
layer2.1.bn1.num_batches_tracked: torch.Size([])
layer2.1.conv2.weight: torch.Size([128, 128, 3, 3])
layer2.1.bn2.weight: torch.Size([128])
```

```
layer2.1.bn2.bias: torch.Size([128])
layer2.1.bn2.running_mean: torch.Size([128])
layer2.1.bn2.running_var: torch.Size([128])
layer2.1.bn2.num_batches_tracked: torch.Size([])
layer3.0.conv1.weight: torch.Size([256, 128, 3, 3])
layer3.0.bn1.weight: torch.Size([256])
layer3.0.bn1.bias: torch.Size([256])
layer3.0.bn1.running_mean: torch.Size([256])
layer3.0.bn1.running_var: torch.Size([256])
layer3.0.bn1.num_batches_tracked: torch.Size([])
layer3.0.conv2.weight: torch.Size([256, 256, 3, 3])
layer3.0.bn2.weight: torch.Size([256])
layer3.0.bn2.bias: torch.Size([256])
layer3.0.bn2.running_mean: torch.Size([256])
layer3.0.bn2.running_var: torch.Size([256])
layer3.0.bn2.num_batches_tracked: torch.Size([])
layer3.0.downsample.0.weight: torch.Size([256, 128, 1, 1])
layer3.0.downsample.1.weight: torch.Size([256])
layer3.0.downsample.1.bias: torch.Size([256])
layer3.0.downsample.1.running_mean: torch.Size([256])
layer3.0.downsample.1.running_var: torch.Size([256])
layer3.0.downsample.1.num_batches_tracked: torch.Size([])
layer3.1.conv1.weight: torch.Size([256, 256, 3, 3])
layer3.1.bn1.weight: torch.Size([256])
layer3.1.bn1.bias: torch.Size([256])
layer3.1.bn1.running_mean: torch.Size([256])
layer3.1.bn1.running_var: torch.Size([256])
layer3.1.bn1.num_batches_tracked: torch.Size([])
layer3.1.conv2.weight: torch.Size([256, 256, 3, 3])
layer3.1.bn2.weight: torch.Size([256])
layer3.1.bn2.bias: torch.Size([256])
layer3.1.bn2.running_mean: torch.Size([256])
layer3.1.bn2.running_var: torch.Size([256])
layer3.1.bn2.num_batches_tracked: torch.Size([])
layer4.0.conv1.weight: torch.Size([512, 256, 3, 3])
layer4.0.bn1.weight: torch.Size([512])
layer4.0.bn1.bias: torch.Size([512])
layer4.0.bn1.running_mean: torch.Size([512])
layer4.0.bn1.running_var: torch.Size([512])
layer4.0.bn1.num_batches_tracked: torch.Size([])
layer4.0.conv2.weight: torch.Size([512, 512, 3, 3])
layer4.0.bn2.weight: torch.Size([512])
layer4.0.bn2.bias: torch.Size([512])
layer4.0.bn2.running_mean: torch.Size([512])
layer4.0.bn2.running_var: torch.Size([512])
layer4.0.bn2.num_batches_tracked: torch.Size([])
layer4.0.downsample.0.weight: torch.Size([512, 256, 1, 1])
layer4.0.downsample.1.weight: torch.Size([512])
layer4.0.downsample.1.bias: torch.Size([512])
layer4.0.downsample.1.running_mean: torch.Size([512])
layer4.0.downsample.1.running_var: torch.Size([512])
layer4.0.downsample.1.num_batches_tracked: torch.Size([])
layer4.1.conv1.weight: torch.Size([512, 512, 3, 3])
layer4.1.bn1.weight: torch.Size([512])
layer4.1.bn1.bias: torch.Size([512])
layer4.1.bn1.running_mean: torch.Size([512])
```

```
layer4.1.bn1.running_var: torch.Size([512])
layer4.1.bn1.num_batches_tracked: torch.Size([])
layer4.1.conv2.weight: torch.Size([512, 512, 3, 3])
layer4.1.bn2.weight: torch.Size([512])
layer4.1.bn2.bias: torch.Size([512])
layer4.1.bn2.running_mean: torch.Size([512])
layer4.1.bn2.running_var: torch.Size([512])
layer4.1.bn2.num_batches_tracked: torch.Size([])
fc.weight: torch.Size([1000, 512])
fc.bias: torch.Size([1000])
```

Load the model from path

The code snippet `model = resnet18(weights=None)` initializes a ResNet-18 convolutional neural network architecture with randomly initialized weights. (https://docs.pytorch.org/vision/0.24/models/generated/torchvision.models.resnet18.html)

In modern versions of PyTorch Torchvision (v0.13+), passing `weights=None` explicitely tells the framework to build the model structure from scratch rather than downloading or loading any pre-trained feature weights (like those trained on ImageNet).

This setup is typically used when you want to train the entire network from scratch on a completely custom dataset. (https://docs.pytorch.org/vision/0.24/models/generated/torchvision.models.resnet18.html)

In [5]:
```
model = resnet18(weights=None) #initializes a ResNet-18 convolutional neural networ
print(model)

# Access the first convolutional layer's weights
print("model.conv1.weight: \n:" , model.conv1.weight)

# Access the final fully connected layer's weights and biases
print("model.fc.weight: \n:" ,model.fc.weight)
print("model.fc.bias: \n:" , model.fc.bias)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=Fal
se)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=Fal
se)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bi
as=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
```

```
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
```

```
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=1000, bias=True)
)
model.conv1.weight:
: Parameter containing:
tensor([[[[-1.3086e-02, -3.1112e-02, -3.6389e-02,  ..., -5.4175e-02,
           5.4395e-02, -2.5005e-02],
          [ 3.8673e-02,  1.3881e-02, -1.7389e-02,  ..., -6.0029e-02,
            1.8984e-02, -2.6445e-02],
          [-1.4626e-02,  5.0921e-03,  1.9642e-02,  ...,  1.2843e-02,
            2.4105e-02, -3.7248e-02],
          ...,
          [-9.4977e-03, -1.4338e-02, -5.4294e-02,  ..., -3.8254e-02,
            4.4293e-02, -2.0731e-03],
          [-1.5389e-02,  3.2904e-02,  1.2909e-02,  ...,  3.6685e-02,
            1.4250e-02,  2.9495e-02],
          [-4.9284e-02,  2.9430e-02,  7.5745e-03,  ..., -3.1274e-04,
            4.1180e-02,  5.2326e-02]],

         [[ 1.7679e-02, -2.7999e-03,  1.7758e-02,  ..., -9.4958e-04,
            7.4765e-03, -7.6752e-03],
          [-2.3257e-03,  3.3647e-04,  1.6102e-02,  ..., -1.8317e-03,
            4.0178e-02,  1.5281e-02],
          [-1.2810e-02,  3.8825e-03,  3.6682e-02,  ...,  1.3285e-02,
           -4.0815e-02,  2.3961e-02],
          ...,
          [-4.2712e-03,  1.8843e-02,  4.9674e-02,  ...,  2.8423e-02,
            2.9430e-02, -1.7722e-02],
          [-3.1959e-02,  2.4426e-02,  3.1414e-04,  ..., -2.3043e-02,
            4.6490e-04,  1.2840e-02],
          [-3.6689e-02,  2.9443e-02, -1.7633e-03,  ...,  3.0678e-02,
            1.9297e-02,  1.1338e-04]],

         [[-1.2344e-02, -1.7498e-02, -1.3338e-02,  ..., -7.0555e-03,
            2.2704e-02,  1.4261e-02],
          [ 9.0282e-03,  9.2836e-04, -3.0686e-02,  ..., -3.8379e-02,
           -1.9619e-04, -2.3578e-02],
          [ 3.4334e-02,  1.6076e-02, -1.5786e-04,  ...,  7.9590e-03,
            1.8516e-02,  2.9628e-03],
          ...,
          [ 2.0154e-02,  3.6974e-02,  8.3311e-03,  ...,  2.4418e-03,
           -4.4408e-02, -1.0242e-03],
          [-1.5331e-02, -1.9499e-02,  5.9082e-03,  ...,  8.8046e-03,
           -6.9272e-03,  3.1608e-02],
          [-2.2746e-02,  6.4436e-03,  1.7021e-02,  ..., -1.1822e-02,
            9.9754e-03, -3.1941e-02]]],


        [[[ 3.3431e-02,  1.4508e-02,  1.5699e-02,  ..., -1.7253e-02,
```

```
  -3.1298e-02, -2.6068e-02],
 [ 1.5179e-02,  1.6135e-02, -2.2674e-03,  ..., -3.0985e-02,
  -1.0993e-02, -1.8818e-02],
 [-1.7652e-02, -3.3454e-02, -3.7808e-03,  ...,  3.9122e-02,
   2.2577e-02, -2.4700e-02],
 ...,
 [-3.5682e-03, -2.6182e-02, -1.4607e-02,  ..., -3.7405e-02,
  -5.4776e-03, -5.1396e-03],
 [-3.7800e-02,  4.6840e-03, -1.6311e-02,  ..., -4.7859e-02,
   1.4744e-02, -6.6287e-04],
 [ 4.1918e-03, -6.1411e-04, -4.3261e-02,  ...,  9.7292e-03,
   1.8748e-02, -1.0631e-02]],

[[ 5.1505e-03,  3.2858e-02,  4.2537e-02,  ...,  9.7442e-04,
   3.5499e-02,  2.9583e-02],
 [ 2.0632e-02,  1.4485e-02,  8.5249e-03,  ..., -3.0861e-02,
   1.2778e-02, -1.5692e-02],
 [ 3.9443e-03, -2.8744e-02,  1.3396e-03,  ...,  2.2632e-02,
  -9.9038e-03, -4.2819e-02],
 ...,
 [ 3.9354e-02,  1.5974e-02,  3.5986e-03,  ...,  2.7315e-03,
   3.6516e-03, -4.7933e-02],
 [ 1.4287e-03, -4.5966e-03, -6.9759e-03,  ..., -3.0708e-02,
  -4.2372e-02,  1.1567e-02],
 [ 3.7950e-02,  1.1240e-03,  4.6108e-02,  ..., -2.0777e-02,
  -3.0915e-02, -1.8039e-02]],

[[ 1.5250e-02, -4.9164e-02, -3.4645e-02,  ...,  3.6571e-02,
   6.3953e-03, -5.7051e-03],
 [ 2.0486e-02, -1.1681e-02, -2.4832e-02,  ...,  9.5862e-03,
   3.5506e-02, -5.1502e-03],
 [-1.5379e-02,  1.3257e-02, -1.8376e-02,  ..., -1.4773e-02,
   6.6286e-03, -7.5662e-03],
 ...,
 [-3.1635e-02,  1.1548e-02,  1.3147e-02,  ..., -1.8167e-02,
  -1.9000e-02, -8.1491e-03],
 [ 1.5846e-03,  3.7215e-02,  1.2012e-02,  ...,  1.7006e-02,
  -2.8085e-02, -1.1328e-02],
 [-3.1765e-02,  2.7048e-03, -3.3641e-02,  ..., -4.3699e-02,
   3.0042e-02,  5.0639e-03]]],


[[[-9.7802e-03, -8.7122e-03, -3.6396e-02,  ...,  4.1968e-03,
  -1.1881e-03,  6.7793e-03],
 [-2.7434e-03,  4.7240e-02,  1.4657e-03,  ..., -3.6633e-03,
   3.1844e-02, -2.8830e-02],
 [ 8.4211e-03, -2.2644e-02, -9.1989e-03,  ..., -1.2080e-02,
   1.6848e-02, -2.2363e-02],
 ...,
 [ 3.3904e-03, -6.5938e-03, -4.7103e-02,  ...,  1.3903e-02,
  -1.0441e-02,  2.0235e-02],
 [ 2.6037e-02, -1.0579e-03, -3.4679e-02,  ...,  4.1253e-02,
   2.9493e-02, -1.6104e-02],
 [ 2.5633e-03, -1.6440e-02,  6.2533e-03,  ..., -1.7470e-02,
   1.3753e-02, -2.4870e-03]],
```

```
 [[-1.8049e-02, -6.5977e-02, -3.8066e-02,  ...,  4.1479e-03,
    8.4795e-03, -9.5176e-03],
  [-2.0926e-02, -3.8062e-02, -1.9736e-02,  ...,  1.3850e-02,
    1.8331e-02, -2.0075e-02],
  [-2.3712e-02, -3.5960e-02, -2.6790e-03,  ...,  1.8628e-02,
   -1.2573e-02, -5.1130e-04],
  ...,
  [-1.9927e-02,  1.9317e-02,  2.9754e-02,  ..., -3.1934e-02,
   -3.7766e-03,  2.8451e-02],
  [ 1.3600e-03, -4.1692e-02,  4.6905e-02,  ...,  2.9793e-02,
   -2.0200e-03, -4.8869e-02],
  [-4.2642e-03,  4.6504e-02,  3.7192e-02,  ..., -1.3980e-03,
    6.8348e-03, -1.5888e-02]],

 [[ 3.3332e-03,  2.7567e-02, -4.7204e-02,  ..., -2.6919e-02,
    8.7295e-03,  1.6939e-02],
  [-1.3619e-02,  1.0928e-02, -1.1339e-02,  ...,  7.6049e-03,
    2.5139e-02, -9.0158e-03],
  [-4.7401e-03,  7.3096e-03, -2.7770e-02,  ...,  1.8326e-02,
   -1.5990e-02,  2.6291e-02],
  ...,
  [ 4.5445e-02, -9.4544e-03,  2.4782e-02,  ..., -2.9071e-02,
   -3.2365e-02,  8.8268e-03],
  [ 4.0765e-02,  2.2570e-02, -1.9113e-02,  ..., -1.8645e-02,
    8.1035e-03, -1.0949e-02],
  [-2.1200e-02,  1.7700e-02, -4.4319e-02,  ...,  5.2274e-03,
    2.6197e-02,  1.6100e-02]]],


...,


[[[ 3.3714e-02,  5.5556e-03,  2.0587e-02,  ...,  2.5200e-02,
    3.8924e-03, -4.0617e-02],
  [ 4.1854e-02,  2.8174e-02, -6.2013e-03,  ...,  2.5031e-03,
   -3.4022e-03, -3.4871e-02],
  [-1.6821e-02, -3.6620e-02, -2.3534e-02,  ..., -1.9622e-02,
    8.2826e-03,  3.7131e-03],
  ...,
  [ 2.0634e-02,  2.2744e-02,  4.5861e-03,  ...,  2.2088e-02,
    2.2627e-03, -1.6724e-02],
  [-3.1468e-02, -3.2112e-02, -1.2265e-02,  ..., -1.7597e-02,
   -2.8272e-02, -1.5313e-02],
  [ 1.3026e-02,  2.0893e-02,  3.9027e-02,  ...,  3.4923e-02,
    9.5520e-03, -1.8129e-02]],

 [[ 8.8904e-03,  3.8515e-02,  4.4425e-02,  ..., -1.3852e-02,
    4.8040e-03,  5.1235e-02],
  [ 3.3025e-03,  2.5527e-02, -9.4300e-03,  ...,  1.4510e-02,
   -3.0307e-02, -3.5022e-02],
  [-3.4314e-02,  1.3796e-02, -9.5422e-03,  ..., -6.8030e-03,
    1.0676e-02,  3.8653e-02],
  ...,
  [ 2.5927e-02, -6.6149e-05,  1.0275e-02,  ...,  3.4913e-02,
    1.4676e-02,  3.8507e-02],
  [-2.4562e-02, -4.3279e-02, -1.0328e-02,  ...,  4.7027e-03,
```

```
   -3.7987e-03,  1.2375e-02],
  [ 2.4334e-02,  1.5566e-02, -2.9046e-02,  ..., -5.4959e-02,
    2.5540e-02,  2.3308e-02]],

 [[-2.9602e-02,  3.7904e-02, -1.9280e-03,  ...,  9.4172e-03,
   -8.8574e-04,  3.4184e-02],
  [-1.7332e-02,  2.1015e-02,  1.2678e-02,  ..., -3.1239e-02,
   -2.9880e-02,  3.0622e-02],
  [ 1.9555e-02,  1.6769e-02, -6.0038e-02,  ..., -1.5786e-02,
   -8.0458e-03, -2.1345e-02],
  ...,
  [-1.4373e-02,  1.9178e-02,  1.5497e-03,  ...,  5.3318e-03,
   -7.8935e-02, -7.5253e-03],
  [ 4.6438e-02,  3.9494e-02, -2.2136e-03,  ...,  1.1705e-02,
    5.7578e-03, -9.7146e-03],
  [-1.8809e-02,  1.0231e-02,  6.0930e-02,  ..., -4.3130e-02,
    2.5284e-02,  1.3115e-02]]],


[[[ 1.8473e-02,  1.7480e-02, -1.1784e-02,  ...,  3.5469e-02,
    1.7634e-02, -2.5749e-02],
  [ 1.8930e-02,  3.1237e-02,  1.7283e-02,  ..., -1.6515e-03,
   -1.8021e-02, -2.1672e-02],
  [-3.6499e-02, -1.3839e-03, -7.3642e-03,  ...,  5.1207e-02,
   -3.3824e-02, -2.6073e-02],
  ...,
  [-2.4507e-02,  1.0007e-02,  4.2950e-02,  ..., -8.0031e-03,
   -1.4661e-02, -2.9270e-02],
  [-1.3482e-02,  2.3737e-03, -2.6394e-02,  ..., -1.9343e-02,
    2.8561e-02,  3.1980e-02],
  [ 2.1348e-02, -1.4224e-02, -1.2017e-02,  ...,  4.0921e-03,
    1.1764e-02, -1.1095e-03]],

 [[-1.3138e-02,  3.3096e-02, -9.2037e-03,  ...,  4.1546e-02,
    2.5493e-02,  2.6701e-02],
  [ 6.4069e-03, -5.4193e-03, -1.0854e-02,  ..., -3.1360e-02,
   -9.0770e-03, -2.9249e-02],
  [-2.7682e-02, -5.4734e-03,  2.0376e-02,  ..., -7.6030e-02,
   -1.8631e-02,  1.6634e-02],
  ...,
  [ 8.2631e-03, -2.4483e-02,  9.4689e-04,  ..., -1.9582e-02,
   -1.1446e-02,  5.0442e-03],
  [-5.7068e-02, -4.0914e-03, -6.1443e-03,  ..., -2.9335e-02,
   -1.2029e-02,  1.0121e-02],
  [-1.5769e-02, -8.2811e-03, -6.3039e-03,  ..., -1.3446e-02,
    1.1988e-02, -3.2937e-02]],

 [[-1.6036e-02,  1.4213e-02,  4.2961e-02,  ..., -8.8144e-03,
    1.7176e-02,  4.1237e-02],
  [ 1.9985e-02,  3.7185e-02,  2.9738e-02,  ...,  2.2794e-02,
    2.8896e-02, -3.9143e-02],
  [ 3.7034e-03, -4.1056e-02, -5.3449e-02,  ..., -2.2375e-02,
    4.3491e-03,  8.5502e-03],
  ...,
  [ 1.6544e-02, -1.0767e-02, -9.2308e-03,  ..., -1.0372e-02,
   -1.7222e-03, -1.9121e-02],
```

```
          [-2.9051e-02,  5.9398e-02,  2.5404e-02,  ...,  2.5333e-02,
            1.2202e-02, -1.2848e-02],
          [-3.1142e-02, -8.7013e-03, -2.6656e-02,  ...,  2.8492e-02,
           -1.6816e-02, -1.6935e-02]]],


        [[[ 2.3377e-02,  1.6590e-02,  2.1225e-02,  ...,  1.9169e-02,
            1.3304e-02, -2.0508e-02],
          [-4.4765e-03, -1.4903e-02, -1.9340e-02,  ...,  2.5715e-02,
            1.3914e-02,  2.5560e-02],
          [-1.0967e-02,  2.7451e-02,  2.2388e-02,  ...,  5.3966e-03,
            2.0108e-02,  1.7914e-02],
          ...,
          [-4.5706e-02, -2.0524e-02,  1.0587e-02,  ...,  2.0916e-02,
            2.9460e-03, -2.7240e-02],
          [-2.9380e-03,  6.5324e-03, -5.4407e-03,  ..., -3.2431e-02,
           -1.4979e-02,  5.0215e-03],
          [-3.3599e-02, -1.2113e-02, -4.7314e-02,  ...,  7.1167e-03,
            3.9054e-03, -1.8187e-02]],

         [[-7.1775e-02, -2.3840e-03,  5.4217e-03,  ...,  3.5302e-02,
            1.1647e-02, -7.6884e-03],
          [-2.4945e-02, -9.5715e-03,  3.9911e-03,  ...,  1.0853e-03,
            3.1263e-02,  1.9822e-02],
          [ 1.9756e-02,  1.1643e-02,  2.3243e-02,  ..., -4.1315e-02,
            1.8440e-03,  5.3246e-03],
          ...,
          [-3.3249e-03, -3.0070e-02,  3.7844e-02,  ...,  1.4901e-02,
           -4.1918e-02, -9.8526e-03],
          [ 3.2752e-02,  3.5931e-04,  1.8155e-02,  ..., -1.9029e-02,
            2.6716e-02,  5.6834e-03],
          [-1.1175e-02, -1.2387e-02,  3.6924e-03,  ..., -1.1679e-03,
           -1.4800e-02, -2.4505e-02]],

         [[ 4.7387e-02,  5.6009e-02, -4.1617e-02,  ...,  1.9190e-03,
           -7.0090e-03,  1.6482e-02],
          [ 1.1483e-02, -2.3147e-03, -1.0276e-02,  ..., -3.1490e-04,
            1.7676e-02,  2.6198e-03],
          [-1.6541e-02, -1.5416e-02, -2.7207e-03,  ..., -1.1480e-02,
            1.3632e-02, -1.2555e-02],
          ...,
          [ 1.8812e-02,  1.4795e-02, -1.2419e-03,  ...,  3.8935e-02,
           -9.3657e-03, -3.0454e-02],
          [-2.8176e-03,  3.4499e-02, -1.9260e-02,  ...,  1.8121e-02,
            2.6551e-02,  1.9333e-03],
          [-5.5164e-03,  7.3086e-03, -2.4567e-02,  ...,  2.6827e-02,
           -3.1277e-03,  6.8055e-02]]]], requires_grad=True)
model.fc.weight:
: Parameter containing:
tensor([[-0.0317,  0.0102,  0.0418,  ...,  0.0017, -0.0421, -0.0211],
        [-0.0054, -0.0069,  0.0378,  ...,  0.0120,  0.0188,  0.0349],
        [ 0.0145, -0.0107, -0.0368,  ...,  0.0419, -0.0330,  0.0325],
        ...,
        [ 0.0222,  0.0330,  0.0063,  ..., -0.0017, -0.0049, -0.0205],
        [ 0.0370, -0.0050, -0.0362,  ...,  0.0046,  0.0302,  0.0294],
        [ 0.0261, -0.0161, -0.0196,  ...,  0.0142, -0.0029, -0.0329]],
```

```
       requires_grad=True)
model.fc.bias:
: Parameter containing:
tensor([-2.4117e-02, -3.7933e-02,  9.7963e-04, -1.0820e-02,  2.6935e-03,
        -4.3112e-02, -4.0845e-02, -2.7943e-02, -1.4312e-02, -4.9923e-03,
        -1.7133e-02, -1.3386e-02,  3.3876e-02, -7.5625e-04, -3.4157e-02,
         3.4147e-02,  2.4332e-02,  2.9141e-02,  3.9899e-02,  4.3666e-02,
        -2.5052e-02,  3.2035e-02, -3.5455e-03, -3.3262e-02,  3.8754e-02,
         2.8115e-03, -3.1822e-02, -9.6254e-03, -2.0181e-02,  1.2686e-02,
         3.3901e-02,  8.1084e-03, -2.8161e-02,  1.9934e-02, -2.0706e-02,
         2.7123e-02, -3.6538e-02, -1.2426e-02,  3.0698e-02,  2.3470e-03,
         1.3638e-02,  3.5717e-02, -3.8454e-02,  3.0099e-02, -1.5004e-02,
        -3.9532e-02,  1.8980e-02,  4.3724e-02,  8.7017e-03, -1.7741e-02,
         4.0132e-02,  2.5724e-02, -1.7622e-02,  1.9998e-02, -9.2812e-03,
         3.6942e-02, -3.5272e-02,  3.5106e-02,  5.0608e-05, -1.5971e-03,
        -2.0890e-02, -2.7758e-02,  3.4425e-02,  3.5607e-02,  2.6190e-02,
         3.6996e-02,  3.5285e-02,  2.0461e-02, -3.0377e-03,  3.2752e-02,
         2.5453e-02,  3.2056e-02, -1.5961e-02, -3.2892e-02, -2.1479e-03,
         3.0227e-02,  2.4508e-02,  3.4686e-02, -3.2594e-02, -5.8296e-03,
         4.2219e-02,  2.9144e-03,  3.6139e-02,  3.5438e-02,  1.2660e-02,
        -3.8065e-02, -3.0929e-02,  3.0298e-02, -2.3649e-02,  3.0673e-02,
         1.1556e-02,  2.8964e-02, -8.2362e-03, -2.6992e-02, -3.7070e-02,
        -3.8203e-02, -3.6611e-02,  5.0848e-03,  3.9825e-02, -1.6918e-02,
         3.6449e-02,  3.4152e-02,  3.6988e-02, -1.5581e-02, -3.4491e-02,
        -2.4126e-03,  4.0758e-02,  2.5801e-02, -2.2247e-02,  4.6956e-04,
        -4.2760e-02,  3.0353e-02, -2.0677e-02, -3.9425e-02,  4.2200e-02,
        -3.9686e-02, -1.9109e-02, -3.6948e-02,  3.0631e-02, -4.2770e-02,
         1.6430e-02,  9.1258e-03,  1.8666e-02,  2.1389e-02,  8.0537e-03,
         9.3363e-03,  7.1609e-03, -1.4121e-02, -1.2703e-02,  1.9672e-02,
         2.3520e-02, -6.8812e-03, -2.2805e-02, -2.5721e-02, -1.7154e-03,
         2.8376e-02, -1.4995e-02, -3.9956e-02, -4.1373e-02, -9.0296e-04,
        -4.0154e-02,  4.5745e-03, -7.6859e-03, -2.8322e-02,  2.1376e-03,
         4.7361e-03,  4.1377e-02, -6.5688e-03,  3.1163e-02, -2.7303e-03,
        -1.6573e-02, -4.3335e-02, -3.4337e-02,  3.0882e-02,  1.0500e-02,
        -8.9397e-03,  9.2963e-03,  3.8369e-02, -3.8378e-02, -3.5385e-02,
         6.1201e-03,  1.0476e-02, -6.6138e-03,  2.8645e-02, -3.1419e-03,
         2.6717e-02, -9.1592e-03,  1.1920e-03, -2.0479e-02, -3.9804e-02,
        -4.9046e-04, -1.7230e-02, -1.0690e-02,  1.8671e-02,  4.0673e-02,
        -3.3376e-02, -2.8610e-02,  1.4348e-02,  1.1179e-02, -1.7704e-02,
         4.9393e-03,  3.8353e-03,  8.8076e-03, -3.0779e-02, -4.0104e-02,
        -1.6398e-02,  2.8576e-02,  2.5871e-02,  1.7129e-02, -8.8134e-03,
        -1.8121e-02, -2.9167e-02, -1.4626e-02, -4.3377e-02, -1.5319e-02,
        -3.5164e-03, -4.3643e-02, -3.1311e-02,  3.5664e-02,  2.9657e-02,
        -1.8056e-02,  4.2979e-02,  2.0364e-02,  6.4372e-03,  2.0411e-02,
         3.9069e-02,  2.6211e-02,  1.5878e-02, -2.0572e-02, -3.5655e-02,
         3.7120e-02,  3.4514e-02,  1.9353e-02, -2.5075e-02,  1.0266e-02,
         2.1584e-02, -5.9859e-03,  1.3424e-02, -3.0735e-02, -4.3302e-02,
        -2.6999e-04,  1.2025e-02, -6.9452e-03, -2.6935e-02, -1.2855e-03,
        -2.2445e-02, -2.0281e-02,  2.4479e-03, -3.3410e-02, -3.9525e-02,
         3.5385e-02,  1.2765e-04,  2.6417e-02, -3.2118e-02, -1.7006e-02,
        -4.2093e-02, -6.0259e-03, -7.0972e-03,  1.6043e-02, -9.3101e-03,
        -1.5537e-02, -1.6476e-03,  3.9265e-03, -1.8971e-02,  1.4587e-02,
         1.2138e-02, -3.6850e-02,  2.0978e-02,  1.5084e-02, -2.6001e-02,
        -2.4886e-02, -9.7027e-03, -3.5997e-02,  2.9726e-03,  3.9254e-02,
         8.5588e-03,  1.3460e-02,  1.5860e-02,  6.5122e-05,  2.4562e-02,
         2.7875e-02, -1.0083e-02, -3.0848e-02, -3.7830e-02,  3.5487e-02,
```

```
 2.7684e-02,  2.7864e-02,  8.2255e-03,  4.1666e-02,  4.9674e-03,
-2.1683e-02, -3.3825e-02, -2.8923e-02, -1.0127e-02, -8.8842e-03,
 4.2261e-02,  4.3752e-02,  2.5294e-02, -3.6889e-02, -1.5303e-02,
 1.1560e-02, -9.9936e-03, -1.1775e-02,  1.8346e-02,  2.7728e-02,
-1.0631e-02,  7.5223e-03, -3.0359e-02, -3.5956e-02,  3.7859e-02,
 4.3767e-02,  2.5341e-02,  4.1024e-03,  4.3200e-02, -4.3022e-02,
 6.4712e-03, -1.9079e-02,  3.9142e-02,  2.4755e-03, -4.4076e-03,
 3.1958e-02,  3.5230e-02, -2.9868e-02, -1.5399e-02,  1.4958e-03,
 7.7926e-03, -6.0195e-03, -3.7798e-02,  3.3347e-02,  1.7290e-02,
-3.0063e-02,  1.1058e-02, -2.9781e-02,  1.8921e-02,  2.4075e-02,
-3.6636e-02,  5.4383e-03,  3.3325e-03, -4.0438e-02,  5.6742e-03,
 6.4007e-03, -1.2074e-02,  2.4905e-02, -2.5758e-02, -1.6563e-03,
 3.0467e-02, -3.4637e-02, -2.2088e-03, -2.5514e-02,  1.4704e-02,
 1.5550e-02,  1.6906e-02,  3.7830e-02,  4.0259e-02,  8.8870e-03,
 3.5739e-02, -2.2575e-02,  4.0584e-02, -8.7720e-03, -7.5378e-03,
 1.6779e-02,  1.4646e-02, -1.6356e-02,  1.3372e-02,  2.8916e-02,
 1.9030e-02, -1.0482e-02,  4.7930e-03, -5.4075e-03, -3.2016e-02,
-2.9992e-02, -6.9195e-03, -3.0331e-02,  1.8318e-02, -3.0350e-02,
-2.0404e-03,  1.5233e-02, -3.5137e-02,  1.1771e-02, -3.3765e-02,
-3.1399e-02,  2.2244e-02, -1.6434e-02, -2.2659e-02, -1.0327e-02,
-4.4141e-02, -2.7299e-04, -4.1546e-02,  3.4935e-02, -2.5936e-02,
 3.5849e-03, -3.5562e-02, -1.0521e-03,  3.2509e-03,  4.3745e-02,
-2.0231e-02,  3.1748e-02,  2.7664e-02,  3.3826e-02, -4.1992e-03,
 9.3350e-03, -3.9497e-02,  1.9879e-02, -2.5770e-03, -3.0591e-02,
 1.0491e-02, -4.2451e-02, -2.0855e-02, -1.1940e-02, -3.2995e-02,
 1.8019e-02,  4.8992e-03, -7.4720e-03,  4.7281e-03,  3.4419e-02,
 2.2742e-02,  1.0580e-02,  7.3134e-03,  2.0333e-02,  1.9470e-02,
 2.3032e-02,  2.1075e-02, -5.7767e-03,  2.9818e-02,  5.1262e-03,
-8.6886e-03,  6.5642e-03,  1.7946e-02, -9.3283e-03, -8.9844e-03,
 1.2400e-02, -4.0898e-02,  1.5983e-02,  9.4346e-03,  1.9510e-02,
-7.3141e-03,  2.4978e-02,  2.3099e-02, -2.6190e-02,  2.1889e-02,
 4.4012e-02, -3.5401e-02,  3.5011e-02,  4.3788e-02,  1.6963e-02,
-2.8752e-02, -5.5295e-03,  1.0810e-02, -2.0480e-02,  1.8403e-02,
-2.8802e-02,  2.6105e-02, -3.7679e-02, -5.4581e-03,  2.1271e-02,
 1.9543e-02, -8.8569e-03, -4.3007e-02,  1.6702e-02,  4.1331e-02,
 8.8604e-03, -1.6685e-02,  4.3630e-02,  2.5360e-02,  1.5615e-03,
-2.1782e-02, -2.7161e-02,  2.3562e-02,  1.9055e-02, -1.9156e-02,
 3.9857e-02, -1.5368e-02,  5.1632e-03,  7.9231e-03, -2.2148e-02,
-4.3949e-02, -1.6331e-02,  4.0527e-02,  1.3421e-02, -1.5041e-02,
 9.1700e-03,  4.0154e-02, -4.0652e-02,  4.1407e-02,  1.8387e-02,
-1.4700e-02,  3.8101e-02,  2.5207e-02, -2.5500e-02, -3.8884e-02,
 1.7042e-02, -2.7773e-02, -4.8815e-03,  3.6693e-02,  2.4061e-02,
 4.0061e-02,  1.5907e-02,  2.6083e-02, -5.1275e-03, -3.8483e-02,
-3.3031e-02, -1.0499e-02, -2.9237e-02,  2.6979e-02,  3.0750e-02,
-2.6906e-02,  3.0771e-02,  1.9476e-02,  2.3910e-02, -1.7738e-02,
 3.7423e-02, -1.7996e-02, -5.4705e-04,  2.0078e-02,  2.8025e-02,
 3.8187e-02, -3.6417e-02,  8.6021e-03, -1.8486e-02,  2.9207e-02,
 2.6874e-02, -3.4264e-02, -2.2227e-02, -3.4211e-02, -3.6697e-02,
-3.5154e-02, -2.0070e-02,  1.9241e-02, -2.5349e-02,  3.8762e-02,
 1.0143e-02,  4.1868e-02, -4.3937e-02,  3.4043e-02,  1.4809e-02,
-3.7942e-02, -9.3666e-04,  2.6658e-02, -1.9103e-04, -3.5837e-02,
-1.0050e-03, -2.8763e-02, -1.3865e-02, -2.7477e-02,  3.6949e-02,
 2.8948e-02, -4.2287e-02, -1.1895e-02, -1.3435e-02,  1.5027e-04,
 6.6853e-03, -1.2845e-03,  1.9734e-02,  2.3675e-02, -4.0662e-02,
-7.3640e-03,  2.0968e-02, -2.5376e-02, -2.5132e-02, -2.3242e-02,
-7.5041e-03,  1.6724e-02,  1.3873e-02,  2.4314e-02, -1.1329e-02,
```

```
-3.7322e-02,  2.0102e-02,  3.5288e-02,  2.7534e-02, -3.3670e-02,
 2.2089e-03,  1.9606e-02, -1.7628e-02,  2.8141e-02,  6.5642e-03,
 2.1986e-02, -4.3573e-02, -3.7474e-02,  2.9580e-02, -1.8226e-02,
 2.9937e-02, -1.6855e-02,  1.6723e-02,  4.0913e-02,  2.4750e-03,
-9.8366e-03, -3.7451e-02, -3.0095e-02, -4.9803e-03, -2.1592e-02,
-1.5901e-02, -2.6885e-02, -2.9261e-02,  3.0110e-02,  2.2930e-02,
 4.0616e-02, -3.7937e-02,  1.2429e-02,  2.2823e-02, -3.9039e-02,
 2.1306e-03,  2.4029e-02, -3.2754e-02,  4.2982e-02, -2.4327e-02,
 4.3583e-02, -2.8492e-03, -2.0034e-02,  1.2236e-02, -2.4791e-02,
 6.2349e-03, -3.9219e-02, -1.6985e-02,  1.7883e-02,  2.0381e-02,
-1.6358e-02,  1.2848e-02,  5.8549e-03,  1.3015e-02,  3.4301e-02,
 4.3225e-02, -7.8592e-03, -1.6479e-02,  9.9257e-03,  1.3167e-02,
 1.2724e-03,  4.2138e-02, -3.9785e-02,  3.9593e-02, -2.7750e-02,
-3.7402e-02,  3.2865e-02,  9.2096e-03,  1.4094e-02,  4.0992e-02,
-1.7410e-02, -3.6079e-02,  4.2521e-02,  7.5902e-03, -7.7598e-03,
 3.0627e-03, -2.0672e-02, -6.3373e-03,  4.2614e-02, -2.2152e-02,
-2.5578e-02,  6.4597e-03, -4.4014e-02,  5.0339e-03,  7.5887e-03,
 1.2718e-02, -1.7058e-02, -2.2254e-02,  2.5091e-02, -2.1790e-02,
-5.4421e-03,  2.2981e-02,  2.0453e-02, -2.4298e-03, -1.5837e-02,
-4.2211e-02, -3.0118e-02,  2.4715e-02,  2.4730e-02, -4.2527e-02,
-1.2538e-02,  3.2136e-03, -1.6710e-03,  2.3268e-02,  9.1679e-03,
 3.1571e-02, -1.5815e-02, -1.1096e-02,  9.8921e-03,  7.9392e-04,
 3.2971e-02, -6.6990e-03,  1.3557e-02, -3.5064e-02,  3.5403e-02,
 4.2664e-02, -1.5800e-02,  4.3398e-02,  4.0640e-02, -3.3751e-02,
 1.3580e-02, -3.8144e-02, -1.8090e-02,  9.3731e-04,  2.0840e-02,
-1.2009e-02,  3.4912e-02, -1.7679e-02, -3.9539e-02,  3.4680e-03,
 3.9684e-02, -3.3325e-02,  2.6713e-03, -3.8206e-02, -3.9972e-02,
-3.1282e-03, -3.8188e-02, -2.8257e-02,  3.2987e-02, -3.8457e-02,
 3.2170e-02,  2.4518e-02,  2.9780e-02,  1.2534e-02, -2.0685e-02,
-1.8907e-02,  2.2013e-02, -1.6981e-02,  4.1264e-02, -2.2175e-02,
 1.0159e-02, -3.2666e-02,  5.2799e-03, -1.0847e-02,  2.8523e-02,
-3.9825e-02, -3.2843e-02,  3.6515e-02, -2.5685e-02,  2.2891e-02,
-2.4747e-02,  5.3063e-03,  4.3668e-02, -3.4650e-02,  3.6823e-02,
-2.8244e-02,  4.2452e-02,  1.3816e-02, -1.5793e-02, -7.1594e-04,
-4.2329e-02,  4.0547e-03,  3.7834e-02,  2.1341e-02, -1.3693e-02,
-3.0579e-02,  7.9177e-03, -3.5001e-02,  3.6562e-03, -2.6743e-02,
 3.7220e-02, -2.7752e-02, -1.7973e-02, -4.1521e-03,  2.5362e-02,
-1.7720e-02,  2.5441e-02, -2.9623e-03, -1.1528e-03,  8.1567e-03,
-1.4192e-02, -8.7125e-03,  3.3447e-02, -3.4173e-02, -2.4116e-02,
 1.4293e-03, -7.6724e-03, -1.9860e-02, -9.7157e-03, -3.2511e-02,
 1.1183e-02,  4.2441e-02, -3.2266e-02,  9.8930e-04,  3.0115e-02,
 4.1097e-02, -3.0117e-02,  1.6868e-03,  6.0134e-03, -5.1189e-03,
 3.2756e-02,  9.7878e-03, -1.6064e-02, -1.9183e-02, -4.0395e-02,
-1.4749e-02, -3.4433e-02,  3.3717e-02,  3.5716e-02, -3.5766e-02,
 3.5245e-03,  4.3115e-02,  3.1519e-02, -3.2467e-02, -1.0816e-02,
-6.2082e-03, -1.1707e-03,  4.0959e-02, -1.3239e-02,  5.0986e-03,
 1.4094e-02,  8.3844e-03,  2.7789e-02,  4.2431e-02,  3.3608e-02,
 2.8772e-02,  4.0176e-02,  6.3305e-03, -7.1043e-04,  1.3736e-02,
 9.9628e-03,  2.4579e-02, -1.7120e-02,  2.8982e-02,  1.6079e-02,
-2.7343e-02,  5.3231e-03,  2.2070e-03, -1.0944e-02, -1.1517e-02,
 4.6747e-03,  5.4891e-03,  2.0638e-02,  1.0116e-02,  3.7790e-03,
-4.4093e-02,  3.0005e-02,  2.8393e-02, -1.8299e-02,  8.7084e-03,
 2.1353e-02, -3.4671e-02,  2.4020e-02,  3.2436e-03,  1.5599e-02,
-1.1943e-02, -2.5489e-02, -1.4920e-02,  4.7223e-03, -3.5522e-02,
-3.2982e-02, -8.6737e-03,  2.7788e-03, -4.1496e-02,  2.4575e-02,
-7.5421e-03, -1.3912e-02, -1.9188e-02, -2.0763e-02,  3.5815e-02,
```

```
 7.2995e-04, -4.0451e-03, -2.8788e-03,  1.0944e-02,  2.5841e-02,
-1.1748e-02,  1.3788e-02, -1.9165e-02,  3.1892e-02, -1.9015e-02,
 2.8376e-02,  1.5461e-03, -3.8323e-02, -1.4483e-02, -3.0304e-02,
 1.4016e-03,  1.6440e-02,  3.6433e-02,  2.3168e-02,  6.6936e-03,
-2.2053e-02, -4.2852e-02,  1.9193e-02,  1.5811e-02, -2.1865e-02,
-4.8377e-03, -2.0948e-02,  8.0655e-03,  7.0774e-03, -2.2731e-02,
 4.3879e-02,  2.0405e-02, -2.5863e-02,  1.0083e-02,  3.8987e-02,
 9.2835e-03, -3.6676e-02,  6.4604e-03,  2.6685e-02, -1.7186e-02,
-2.5990e-02,  2.1052e-02,  4.2546e-02, -1.4339e-03, -4.2522e-02,
 1.8104e-02, -1.9512e-02,  1.9702e-02, -1.6975e-02,  1.6440e-02,
-7.9617e-03,  1.3070e-02,  4.1806e-02,  3.2236e-02,  1.6571e-02,
 9.1078e-03,  3.6698e-02,  1.0057e-02, -1.4100e-02,  2.4984e-02,
-1.5487e-02,  3.0544e-02,  1.0943e-02, -4.2081e-02,  2.1963e-02,
 4.4177e-02, -1.6660e-02, -1.2521e-03,  2.9780e-02, -3.1117e-02,
 2.4003e-03, -7.6231e-03, -4.3868e-02,  1.5720e-03,  2.2478e-02,
-8.9548e-03,  1.5770e-03, -4.2172e-02, -2.1309e-02,  3.5271e-02,
 9.2696e-03, -3.6379e-02, -1.7342e-02,  5.1165e-03, -3.8103e-02,
-2.7791e-02, -3.8910e-02, -4.6293e-03, -1.0099e-03, -2.9652e-02,
 4.0334e-02,  5.8412e-03, -3.1735e-02,  3.9167e-02, -4.3619e-02,
 2.2345e-02,  2.3266e-02, -1.0621e-03, -3.5152e-02,  1.9975e-02,
-3.6231e-03, -1.9427e-02, -5.8426e-03, -1.5828e-02, -5.2194e-03,
 3.4676e-02, -3.4097e-02, -1.8852e-02, -9.0134e-03, -7.2298e-03,
-2.3476e-02, -1.4194e-02, -8.1652e-03,  2.9315e-02,  1.0901e-02,
-2.9688e-02, -1.3316e-02,  9.5899e-03, -4.6760e-03, -8.2487e-03,
-3.4587e-02, -2.7070e-03,  2.8827e-02, -1.9739e-02,  2.8208e-02,
-9.0963e-03,  3.4196e-02,  3.3862e-02,  2.2745e-02,  1.0312e-02,
-3.7694e-02, -4.1191e-02, -2.2713e-02, -2.2979e-02, -3.8086e-02,
 2.8223e-02, -7.8154e-03,  1.4612e-02, -7.1470e-03,  4.0933e-02,
-4.1522e-02,  3.5925e-02,  2.9106e-02, -3.5312e-02, -1.0882e-02,
-1.7263e-02,  1.3144e-03,  4.8318e-03, -3.4144e-02,  2.9715e-02,
 1.2130e-02,  1.9185e-02, -3.6789e-02,  1.6646e-02, -2.9841e-02,
-1.4231e-02,  2.1365e-02,  1.3357e-02, -2.0553e-02, -8.5419e-03,
-2.4267e-02,  1.2274e-02, -4.3875e-02, -3.8952e-02,  5.2353e-03,
-1.4806e-04, -2.3930e-02, -5.3433e-03, -2.7817e-02,  1.5436e-02,
-2.6638e-02, -3.4437e-02, -2.3703e-02,  2.1001e-02,  2.1898e-02],
requires_grad=True)
```

In [6]:

```
PATH = r'D:\Teaching\UL\AI\Lab\resnetModel\resnet18.pth'
#LOAD model parameters in dictionary
model.load_state_dict(torch.load(PATH))

print(model)
# Access the first convolutional layer's weights
print("model.conv1.weight: \n:" , model.conv1.weight)

# Access the final fully connected layer's weights and biases
print("model.fc.weight: \n:" , model.fc.weight)
print("model.fc.bias: \n:" , model.fc.bias)
```

```
ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=Fal
se)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=Fal
se)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bi
as=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
```

```
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
```

```
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=1000, bias=True)
)
model.conv1.weight:
: Parameter containing:
tensor([[[[-1.0419e-02, -6.1356e-03, -1.8098e-03,  ...,  5.6615e-02,
            1.7083e-02, -1.2694e-02],
          [ 1.1083e-02,  9.5276e-03, -1.0993e-01,  ..., -2.7124e-01,
           -1.2907e-01,  3.7424e-03],
          [-6.9434e-03,  5.9089e-02,  2.9548e-01,  ...,  5.1972e-01,
            2.5632e-01,  6.3573e-02],
          ...,
          [-2.7535e-02,  1.6045e-02,  7.2595e-02,  ..., -3.3285e-01,
           -4.2058e-01, -2.5781e-01],
          [ 3.0613e-02,  4.0960e-02,  6.2850e-02,  ...,  4.1384e-01,
            3.9359e-01,  1.6606e-01],
          [-1.3736e-02, -3.6746e-03, -2.4084e-02,  ..., -1.5070e-01,
           -8.2230e-02, -5.7828e-03]],

         [[-1.1397e-02, -2.6619e-02, -3.4641e-02,  ...,  3.2521e-02,
            6.6221e-04, -2.5743e-02],
          [ 4.5687e-02,  3.3603e-02, -1.0453e-01,  ..., -3.1253e-01,
           -1.6051e-01, -1.2826e-03],
          [-8.3730e-04,  9.8420e-02,  4.0210e-01,  ...,  7.0789e-01,
            3.6887e-01,  1.2455e-01],
          ...,
          [-5.5926e-02, -5.2239e-03,  2.7081e-02,  ..., -4.6178e-01,
           -5.7080e-01, -3.6552e-01],
          [ 3.2860e-02,  5.5574e-02,  9.9670e-02,  ...,  5.4636e-01,
            4.8276e-01,  1.9867e-01],
          [ 5.3051e-03,  6.6938e-03, -1.7254e-02,  ..., -1.4822e-01,
           -7.7248e-02,  7.2183e-04]],

         [[-2.0315e-03, -9.1617e-03,  2.1209e-02,  ...,  8.9177e-02,
            3.3655e-02, -2.0102e-02],
          [ 1.5398e-02, -1.8648e-02, -1.2591e-01,  ..., -2.5342e-01,
           -1.2980e-01, -2.7975e-02],
          [ 9.8454e-03,  4.9047e-02,  2.1699e-01,  ...,  3.4872e-01,
            1.0433e-01,  1.8413e-02],
          ...,
          [-2.8356e-02,  1.8404e-02,  9.8647e-02,  ..., -1.1740e-01,
           -2.5760e-01, -1.5451e-01],
          [ 2.0766e-02, -2.6286e-03, -3.7825e-02,  ...,  2.4141e-01,
            2.4345e-01,  1.1796e-01],
          [ 7.4684e-04,  7.7677e-04, -1.0050e-02,  ..., -1.4865e-01,
           -1.1754e-01, -3.8350e-02]]],


        [[[-4.4154e-03, -4.0645e-03,  3.1589e-03,  ..., -3.7026e-02,
```

```
  -2.5158e-02, -4.7945e-02],
 [ 5.1310e-02,  5.3402e-02,  8.0436e-02,  ...,  1.4480e-01,
   1.4287e-01,  1.2312e-01],
 [-7.3337e-03,  2.1755e-03,  3.7580e-02,  ...,  6.1517e-02,
   8.0324e-02,  1.1715e-01],
 ...,
 [-2.6754e-02, -1.2297e-01, -1.3653e-01,  ..., -1.4068e-01,
  -1.1155e-01, -4.9556e-02],
 [ 2.3524e-02, -1.7288e-02, -1.1122e-02,  ..., -1.8826e-02,
  -2.3320e-02, -2.9474e-02],
 [ 2.8689e-02,  2.1659e-02,  4.7888e-02,  ...,  2.5498e-02,
   3.5346e-02,  1.1280e-02]],

[[ 4.6919e-04,  1.2153e-02,  4.2035e-02,  ...,  4.6403e-02,
   4.0423e-02, -1.4439e-02],
 [ 4.3463e-02,  6.8779e-02,  1.3268e-01,  ...,  2.8606e-01,
   2.6905e-01,  2.0935e-01],
 [-5.7621e-02, -2.2642e-02,  3.0547e-02,  ...,  1.3763e-01,
   1.6538e-01,  1.7946e-01],
 ...,
 [-1.0816e-01, -2.5227e-01, -2.9742e-01,  ..., -2.8503e-01,
  -2.1493e-01, -1.0320e-01],
 [ 4.0709e-02, -3.2771e-02, -6.3450e-02,  ..., -9.2360e-02,
  -6.9876e-02, -4.9841e-02],
 [ 8.2942e-02,  8.7580e-02,  1.0111e-01,  ...,  5.2714e-02,
   6.0968e-02,  4.1198e-02]],

[[-1.6391e-02, -1.3870e-02,  5.2810e-03,  ...,  4.3698e-02,
   2.2707e-02, -4.5983e-02],
 [ 3.3202e-02,  4.2014e-02,  9.3500e-02,  ...,  2.6162e-01,
   2.2970e-01,  1.6694e-01],
 [-4.5987e-02, -1.6365e-02,  2.6811e-02,  ...,  1.4951e-01,
   1.3216e-01,  1.3579e-01],
 ...,
 [-7.2129e-02, -1.8902e-01, -2.3389e-01,  ..., -1.9038e-01,
  -1.5609e-01, -7.5974e-02],
 [ 5.1161e-02, -2.5815e-02, -6.9357e-02,  ..., -5.8999e-02,
  -6.1550e-02, -4.4555e-02],
 [ 1.1174e-01,  7.8979e-02,  6.5849e-02,  ...,  3.1617e-02,
   2.5221e-02,  7.4257e-03]]],


[[[-7.0826e-08, -6.4306e-08, -7.3806e-08,  ..., -9.8000e-08,
   -1.0905e-07, -8.3421e-08],
 [-6.1125e-09,  2.0613e-09, -8.0922e-09,  ..., -4.9840e-08,
  -4.3836e-08, -3.0538e-09],
 [ 7.1953e-08,  7.5616e-08,  5.9282e-08,  ..., -9.7509e-09,
  -1.0951e-09,  4.2442e-08],
 ...,
 [ 9.5889e-08,  1.0039e-07,  7.9817e-08,  ..., -1.7491e-08,
  -4.7666e-08, -1.3265e-08],
 [ 1.2904e-07,  1.4762e-07,  1.7477e-07,  ...,  1.3233e-07,
   1.0628e-07,  9.3316e-08],
 [ 1.2558e-07,  1.3644e-07,  1.8431e-07,  ...,  2.1399e-07,
   1.7710e-07,  1.7166e-07]],
```

```
 [[-1.2690e-07, -9.6139e-08, -1.0372e-07,  ..., -1.1808e-07,
   -1.3309e-07, -1.0820e-07],
  [-5.7412e-08, -2.5055e-08, -3.0115e-08,  ..., -7.2922e-08,
   -6.7022e-08, -2.2574e-08],
  [ 2.1813e-08,  4.8608e-08,  3.1222e-08,  ..., -1.8694e-08,
   -7.9591e-09,  3.9750e-08],
  ...,
  [ 5.6013e-08,  7.5526e-08,  4.4496e-08,  ..., -4.4128e-08,
   -5.9930e-08, -1.8247e-08],
  [ 7.7614e-08,  9.8348e-08,  1.0455e-07,  ...,  6.3272e-08,
    4.1781e-08,  4.5901e-08],
  [ 5.9834e-08,  7.1006e-08,  9.0437e-08,  ...,  1.1654e-07,
    8.7550e-08,  9.8837e-08]],

 [[-4.3810e-08,  1.3270e-08,  7.8275e-09,  ..., -5.8804e-09,
   -2.6217e-08, -1.5649e-08],
  [ 4.1700e-08,  1.0778e-07,  1.0946e-07,  ...,  7.6403e-08,
    7.1450e-08,  9.7615e-08],
  [ 1.0436e-07,  1.6586e-07,  1.5933e-07,  ...,  1.3517e-07,
    1.3487e-07,  1.6449e-07],
  ...,
  [ 9.8763e-08,  1.5072e-07,  1.2547e-07,  ...,  6.8316e-08,
    6.8382e-08,  1.1367e-07],
  [ 9.1435e-08,  1.3576e-07,  1.3793e-07,  ...,  1.1678e-07,
    1.1723e-07,  1.4394e-07],
  [ 6.2183e-08,  8.8184e-08,  1.0456e-07,  ...,  1.3941e-07,
    1.3333e-07,  1.5844e-07]]],


...,


[[[-6.1896e-02, -3.0206e-02,  1.9225e-02,  ...,  4.3665e-02,
   -2.2114e-02, -4.2214e-02],
  [-3.8061e-02,  6.0774e-03,  4.5797e-02,  ...,  9.6029e-02,
    5.9254e-02,  2.9958e-02],
  [-2.9672e-02,  2.7766e-03,  2.0457e-02,  ...,  5.9828e-02,
    4.1422e-02,  2.3134e-02],
  ...,
  [ 1.1916e-02,  4.5701e-02,  4.4892e-02,  ...,  4.7419e-02,
    2.2274e-02, -5.4993e-03],
  [-3.2468e-02, -1.2210e-02,  2.2023e-02,  ...,  5.8061e-02,
   -7.5033e-03, -5.9736e-02],
  [-4.3314e-02, -2.8162e-02, -5.9126e-03,  ...,  8.8460e-02,
    8.4406e-03, -5.0019e-02]],

 [[-6.1292e-02, -1.4004e-02,  1.7229e-02,  ...,  1.8349e-02,
   -3.2708e-02, -4.1060e-02],
  [-3.1506e-02,  2.4460e-02,  4.5516e-02,  ...,  6.6806e-02,
    4.6687e-02,  3.3248e-02],
  [-3.2216e-02,  2.0718e-02,  2.3343e-02,  ...,  3.5265e-02,
    3.6478e-02,  3.1291e-02],
  ...,
  [ 1.7739e-02,  6.1040e-02,  4.8247e-02,  ...,  3.7785e-02,
    2.8894e-02,  1.3984e-02],
  [-1.0890e-02,  2.2079e-02,  4.2737e-02,  ...,  6.0247e-02,
```

```
    1.6197e-02, -1.2493e-02],
   [-2.2284e-02,  1.3220e-02,  3.0897e-02,  ...,  1.0403e-01,
    4.0119e-02, -5.3310e-03]],

  [[-8.5322e-02, -4.2603e-02,  6.8145e-03,  ...,  3.0751e-02,
   -3.4818e-02, -4.9945e-02],
   [-2.9215e-02,  1.8165e-02,  5.1092e-02,  ...,  9.0200e-02,
    5.3438e-02,  4.0169e-02],
   [-3.9932e-02, -1.1100e-03,  9.6176e-03,  ...,  2.4114e-02,
    2.6298e-02,  2.5489e-02],
   ...,
   [-3.1890e-03,  3.0454e-02,  1.6316e-02,  ...,  5.5054e-03,
   -6.2689e-03, -8.4638e-03],
   [-2.2995e-02, -2.8211e-03,  2.3203e-02,  ...,  3.5888e-02,
   -1.4296e-02, -3.2419e-02],
   [-9.8894e-03,  7.0542e-03,  1.0659e-02,  ...,  7.0495e-02,
    1.2996e-02, -8.3417e-03]]],


 [[[-7.8699e-03,  1.9911e-02,  3.4208e-02,  ...,  2.8694e-02,
    1.2820e-02,  1.8142e-02],
   [ 8.7942e-03, -3.2875e-02, -3.5713e-02,  ...,  7.2533e-02,
    4.5889e-02,  5.2383e-02],
   [-3.6122e-02, -1.1878e-01, -1.3767e-01,  ...,  3.3811e-02,
    3.7806e-02,  2.6944e-02],
   ...,
   [ 1.7322e-02,  3.9589e-03, -8.2269e-03,  ...,  2.7543e-03,
    1.8313e-02,  1.6057e-02],
   [-9.5007e-04,  1.6428e-02,  1.7156e-02,  ...,  3.3672e-03,
    2.2857e-02,  6.5783e-04],
   [ 6.1727e-03,  2.7145e-02,  1.4340e-02,  ...,  7.5867e-03,
    1.8770e-02,  1.5624e-02]],

  [[-1.3423e-02, -5.0696e-04,  8.0959e-03,  ..., -6.0963e-03,
    9.2341e-03,  1.5751e-02],
   [-1.8343e-02, -6.7982e-02, -7.0685e-02,  ...,  2.9855e-02,
    2.6264e-02,  2.3773e-02],
   [-5.4359e-02, -1.4663e-01, -1.6211e-01,  ...,  1.1781e-02,
    3.2477e-02,  1.1980e-02],
   ...,
   [ 8.3686e-04, -1.7564e-02, -1.9535e-02,  ..., -4.1382e-03,
    2.4658e-02,  1.2893e-02],
   [-6.3183e-04,  1.1788e-02,  2.4810e-02,  ...,  6.1105e-03,
    3.9210e-02,  9.6696e-03],
   [-7.1831e-03,  6.6918e-03,  5.2723e-03,  ..., -7.6077e-03,
    2.7253e-02,  1.7735e-02]],

  [[-2.3753e-04, -4.9343e-03,  2.2991e-03,  ..., -4.7958e-02,
   -2.6154e-02, -2.3525e-02],
   [-3.3053e-04, -5.1502e-02, -5.9977e-02,  ..., -1.7369e-02,
   -2.3337e-02, -3.7312e-02],
   [-2.2674e-02, -9.9412e-02, -1.1176e-01,  ..., -1.1725e-02,
   -8.3744e-03, -4.0615e-02],
   ...,
   [ 1.1437e-02, -8.0313e-03, -1.4955e-03,  ..., -3.4133e-02,
   -8.7267e-03, -2.3526e-02],
```

```
          [ 2.9522e-03,  6.7770e-04,  1.9933e-02,  ..., -2.2002e-02,
            1.4814e-02, -1.4487e-02],
          [-1.9085e-02, -2.9430e-02, -2.3284e-02,  ..., -4.8587e-02,
           -1.3049e-02, -2.4368e-02]]],


        [[[-3.6296e-02,  7.1996e-03,  1.9100e-02,  ...,  1.9602e-02,
            1.4870e-02, -1.7298e-02],
          [-1.1061e-02,  8.5665e-02,  1.2667e-01,  ...,  1.3744e-02,
           -5.5036e-05, -3.0162e-02],
          [ 1.1322e-01,  1.8634e-01,  5.0658e-02,  ..., -1.7333e-01,
           -7.2041e-02, -6.2474e-02],
          ...,
          [-5.3062e-02, -2.5781e-01, -2.6747e-01,  ...,  2.6781e-01,
            1.4344e-01,  5.5145e-02],
          [-2.1009e-02, -2.9969e-02,  1.0245e-01,  ...,  2.0843e-01,
           -4.1518e-03, -3.8118e-02],
          [-2.2155e-02,  1.2380e-02,  8.4302e-02,  ..., -4.4992e-02,
           -1.4687e-01, -9.0890e-02]],

         [[-5.3969e-03,  3.2799e-02,  1.5486e-02,  ..., -7.7451e-03,
            3.0229e-03,  1.1216e-03],
          [ 6.1723e-02,  1.4899e-01,  1.4645e-01,  ..., -2.8897e-02,
           -2.0227e-02, -9.1878e-03],
          [ 1.6146e-01,  2.0886e-01, -2.5589e-02,  ..., -2.7278e-01,
           -1.0735e-01, -6.2971e-02],
          ...,
          [-1.3723e-01, -4.0863e-01, -3.8551e-01,  ...,  4.0846e-01,
            2.6202e-01,  1.3491e-01],
          [-5.9388e-02, -6.1187e-02,  1.4197e-01,  ...,  3.5780e-01,
            9.0893e-02, -1.7392e-03],
          [ 7.8613e-03,  5.8403e-02,  1.5339e-01,  ...,  4.7045e-02,
           -1.0095e-01, -9.7920e-02]],

         [[-5.6799e-03,  1.3425e-02, -2.6461e-02,  ...,  4.4881e-03,
            2.0666e-03,  1.3902e-02],
          [ 6.5943e-03,  4.5181e-02,  6.0260e-02,  ...,  1.4368e-02,
           -5.0725e-03,  4.0505e-03],
          [ 5.5257e-02,  1.2397e-01,  4.3193e-02,  ..., -1.4486e-01,
           -7.4489e-02, -5.7533e-02],
          ...,
          [-3.1513e-02, -1.6334e-01, -1.5795e-01,  ...,  2.2904e-01,
            1.2017e-01,  7.1998e-02],
          [-1.0456e-02, -1.1248e-03,  8.4582e-02,  ...,  1.5748e-01,
            2.2142e-02, -1.0083e-02],
          [-4.8639e-03, -5.0065e-03,  3.6341e-02,  ..., -2.4361e-02,
           -7.1195e-02, -6.6788e-02]]]], requires_grad=True)
model.fc.weight:
: Parameter containing:
tensor([[-0.0185, -0.0705, -0.0518,  ..., -0.0390,  0.1735, -0.0410],
        [-0.0818, -0.0944,  0.0174,  ...,  0.2028, -0.0248,  0.0372],
        [-0.0332, -0.0566, -0.0242,  ..., -0.0344, -0.0227,  0.0197],
        ...,
        [-0.0103,  0.0033, -0.0359,  ..., -0.0279, -0.0115,  0.0128],
        [-0.0359, -0.0353, -0.0296,  ..., -0.0330, -0.0110, -0.0513],
        [ 0.0021, -0.0248, -0.0829,  ...,  0.0417, -0.0500,  0.0663]],
```

```
       requires_grad=True)
model.fc.bias:
: Parameter containing:
tensor([-2.6341e-03,  3.0005e-03,  6.5581e-04, -2.6909e-02,  6.3637e-03,
         1.3260e-02, -1.1178e-02,  2.0639e-02, -3.6373e-03, -1.2325e-02,
        -1.2629e-02, -7.2057e-03, -1.9321e-02, -2.4960e-02, -1.1885e-02,
        -8.3259e-03, -9.5745e-03, -1.6658e-02,  9.1804e-03, -1.5354e-02,
         7.1358e-03,  3.0737e-02,  1.3239e-02, -7.7528e-03,  4.7448e-03,
         1.1175e-02,  1.5949e-02, -1.6712e-02, -1.0130e-03, -3.7167e-03,
         6.5269e-03, -1.2041e-02,  9.0427e-03, -8.3279e-04,  8.8647e-03,
        -2.6307e-02, -1.4588e-02,  2.9433e-03,  2.9718e-03, -1.9125e-02,
        -4.7922e-03,  1.3828e-02,  9.8802e-03, -1.8417e-02,  1.9734e-02,
         1.6941e-03,  1.2420e-02, -5.5842e-03, -1.0612e-02,  3.9847e-04,
         4.2733e-03, -1.3298e-02,  2.0661e-02,  1.6963e-02,  2.7952e-03,
         7.4214e-04,  1.3168e-02,  3.2213e-03,  1.0458e-02,  1.6511e-02,
         9.1717e-04,  3.9388e-03, -5.6534e-03,  1.9372e-02,  7.5238e-03,
         1.3437e-02, -1.3185e-02, -1.0026e-02,  7.1920e-03, -2.3166e-03,
        -1.8895e-02,  1.2519e-02,  1.9583e-03,  7.3836e-03, -9.6664e-03,
         2.0189e-02,  7.6652e-03,  1.8529e-02,  1.5710e-02,  1.8582e-02,
        -6.9314e-03,  1.7090e-02,  9.1268e-03, -3.8876e-02, -2.4116e-02,
        -6.8715e-03, -1.1648e-02,  7.8817e-03,  1.8046e-03,  2.8480e-02,
        -1.9379e-02, -1.6295e-02,  1.0468e-02, -1.3027e-02, -8.4211e-03,
        -2.9210e-02, -2.4856e-03, -8.7141e-03, -1.6397e-02, -9.3054e-03,
        -1.5931e-02, -2.6346e-02, -1.1091e-03,  2.2589e-02,  2.1387e-03,
        -2.3212e-02, -1.4085e-02, -5.6224e-03, -2.0090e-02, -3.0284e-02,
        -4.9574e-02,  2.3283e-02,  1.4954e-02, -7.7501e-03, -3.9482e-03,
        -3.7629e-02, -2.4220e-02, -1.0194e-02, -7.7038e-03, -4.1312e-03,
        -2.9553e-03, -6.2174e-03, -1.2076e-02, -7.0168e-03, -3.8948e-03,
        -1.6953e-02, -2.4585e-02,  5.5353e-03, -8.3370e-03, -7.0759e-03,
        -2.4023e-02, -6.3686e-03,  7.3420e-04,  5.2883e-03, -2.2181e-02,
        -2.6972e-02, -1.7990e-02, -1.6393e-02,  2.1485e-03, -1.6122e-02,
        -1.6112e-02,  6.5931e-03, -2.0045e-02,  6.4149e-03, -1.2601e-02,
        -7.6238e-03,  1.1411e-02, -4.5084e-02, -9.2018e-03, -1.5563e-02,
        -1.3590e-02, -1.4374e-03, -1.9466e-02,  2.0737e-02, -1.0476e-02,
         6.3229e-03,  8.3229e-03, -1.0791e-02, -1.8903e-02,  5.8624e-03,
        -2.0189e-03,  3.2436e-02,  4.0581e-02, -4.0820e-05,  1.0886e-02,
        -1.6544e-02, -5.3365e-04, -2.2903e-02,  4.6295e-03, -4.8402e-03,
         1.0187e-02,  1.7954e-02,  4.8211e-03,  6.1831e-03,  1.4419e-02,
        -1.2094e-02, -8.7460e-03,  1.9488e-03,  1.4685e-02,  1.2464e-02,
         7.0523e-03, -4.1783e-03,  1.2048e-02, -2.0199e-02,  9.9144e-03,
         1.3978e-02, -1.0321e-03,  5.7394e-03,  1.4019e-03,  6.0113e-04,
        -5.5790e-04,  2.4424e-02,  2.3076e-02, -1.4610e-02,  1.1185e-02,
         3.4608e-02,  1.6944e-02,  4.3295e-03, -2.5606e-02,  1.2279e-02,
        -2.5810e-02,  8.5365e-03,  2.0437e-02,  2.2557e-02,  2.2966e-02,
         8.8420e-03, -1.3894e-02,  3.8719e-03, -9.3046e-03,  2.3220e-02,
        -1.4949e-02,  6.9258e-03,  5.0070e-03, -1.7302e-02,  1.0364e-03,
        -1.0223e-02, -9.6949e-03,  3.4534e-02,  6.1337e-03,  1.1582e-02,
        -2.0529e-02, -2.1956e-02,  5.3109e-03,  3.4101e-02, -5.8079e-03,
         2.9406e-02, -8.7954e-03, -5.2505e-03, -1.9088e-02,  3.0350e-02,
         1.8445e-02, -2.1225e-02,  1.8432e-02,  1.3832e-02,  1.7848e-02,
        -4.4762e-03,  3.5858e-02,  2.1762e-02,  1.0880e-02,  4.0255e-02,
        -2.0049e-03, -3.0348e-03,  9.3293e-03, -1.6304e-02,  9.6253e-04,
         1.8673e-02, -1.6567e-02,  1.4964e-02, -3.7206e-03, -7.6734e-03,
        -7.9254e-06,  3.9732e-03, -9.5979e-03, -1.6833e-02,  5.8524e-05,
        -6.4126e-03,  8.2977e-03,  4.8207e-03, -1.1467e-03,  4.8869e-03,
         1.7349e-02,  3.9222e-03, -7.8080e-03,  1.6051e-02,  9.8802e-03,
```

```
-1.0144e-02,  2.0912e-02, -6.3203e-03, -2.3139e-02,  1.1646e-03,
 2.2468e-02, -6.6953e-03,  1.8311e-02,  1.4623e-02, -1.1654e-02,
-1.4306e-02,  1.2974e-02, -9.6865e-03, -6.2351e-03,  1.3180e-02,
 6.7543e-03,  4.6418e-02, -2.7962e-02, -1.5111e-02,  2.8716e-02,
 9.1991e-03, -5.3710e-03, -6.0361e-03, -7.2140e-03, -9.2421e-03,
 1.8536e-03, -3.1078e-03, -8.4004e-03, -1.6766e-02,  4.0936e-03,
 6.2426e-03, -1.2470e-03, -1.2919e-02,  3.5819e-03,  1.1006e-02,
-1.3282e-02,  2.6395e-03,  8.9953e-03,  6.5421e-03, -1.2031e-02,
 1.7149e-02,  1.7949e-02, -1.0581e-02, -2.6962e-02, -1.3564e-02,
-9.7173e-03, -2.1176e-03,  3.5370e-02,  1.8392e-02,  2.6676e-02,
-1.0594e-03, -3.3949e-03, -4.8838e-03,  1.3427e-02, -1.3948e-02,
-1.9559e-02, -2.3295e-02, -3.7834e-02, -1.4637e-02, -2.1323e-02,
-3.0952e-02, -3.0822e-02,  1.9438e-03,  2.8637e-03, -2.1198e-02,
 1.0448e-02, -1.1316e-02, -4.2609e-03,  2.2647e-02, -1.2867e-02,
-1.1018e-02,  1.2336e-02, -2.0057e-02, -2.1837e-02, -6.8067e-03,
-1.0488e-02, -2.6298e-02, -9.9579e-03, -1.2966e-02, -2.4832e-03,
-7.0940e-03,  1.7997e-02, -6.4257e-03,  7.9069e-03, -1.2287e-02,
 9.8176e-03,  2.6674e-03, -2.1524e-02,  2.7511e-03, -9.2075e-03,
-1.7541e-02, -1.7103e-03, -1.4588e-02,  4.4247e-03,  3.4405e-02,
 1.2725e-02,  3.0885e-02,  1.3090e-03, -7.0084e-05, -2.3165e-03,
-3.7989e-03, -1.1148e-02,  1.7210e-02, -6.7575e-03, -1.3694e-04,
-8.9166e-03, -1.6281e-02, -4.4920e-03,  1.1332e-02, -1.5909e-03,
-8.8193e-03, -4.9399e-03,  4.5732e-03, -1.0949e-02,  1.2890e-02,
-6.6586e-03, -2.5605e-03,  2.7965e-03,  1.1225e-02, -2.2055e-02,
-3.9271e-03, -6.6467e-03, -1.8840e-02, -2.1687e-02, -7.4066e-04,
-2.7281e-02,  5.0448e-03, -2.0709e-02, -3.4103e-02, -2.2374e-02,
-1.6656e-02, -2.7916e-02, -9.8977e-03,  5.5252e-03,  1.6013e-02,
-1.4895e-02,  3.5091e-03,  9.0003e-03, -8.3982e-03, -3.7479e-02,
 2.0727e-02, -5.8799e-03,  9.1768e-03, -2.0297e-02, -7.3148e-03,
-1.6966e-03, -1.4029e-03,  3.2229e-03,  2.9212e-02,  1.2487e-02,
-2.0100e-02,  2.1170e-02, -2.5300e-02,  3.1815e-02, -1.0645e-03,
-1.0449e-02, -2.3419e-02,  1.4564e-02,  2.1245e-02,  1.6530e-02,
-3.2436e-03, -2.0437e-02, -3.6982e-02, -8.7213e-03,  5.4575e-03,
 1.1048e-03,  2.2012e-03,  2.9512e-03, -5.9939e-05,  5.5785e-04,
-3.6906e-03,  5.3763e-03, -2.4765e-02,  9.2729e-03,  9.6081e-03,
 9.1647e-03,  9.0880e-03,  7.4842e-03, -1.1946e-02,  2.1395e-02,
 2.7922e-02,  1.4692e-02, -2.4958e-03,  2.8887e-02,  1.3422e-02,
 1.7173e-03,  2.5018e-03, -2.1253e-02, -8.2424e-04,  4.2183e-03,
 8.5981e-03,  1.8735e-02,  8.5622e-03, -8.8255e-03,  1.7462e-02,
-1.3693e-02,  2.1955e-03,  1.0772e-02,  2.8693e-03,  3.1032e-02,
 8.5460e-03, -1.4198e-02, -2.2472e-03,  1.8740e-02, -1.2905e-02,
 4.0370e-02, -7.7538e-04,  1.8671e-03,  7.2793e-03, -2.6508e-02,
-1.7609e-02, -2.4142e-02,  2.9577e-03, -1.5917e-02,  1.6273e-03,
 1.1132e-02,  1.4574e-02, -8.1919e-03, -7.6581e-03, -1.8452e-02,
-9.0419e-03,  4.0883e-03,  4.4482e-02, -2.3664e-02, -5.2547e-03,
-1.9529e-02,  3.2860e-03,  5.4667e-03, -4.9558e-03,  7.6805e-03,
-3.3026e-03, -2.6248e-03, -1.1094e-02,  2.3922e-02,  1.8079e-02,
-1.8135e-02,  5.2204e-03, -1.3559e-02,  1.9448e-02,  1.0981e-02,
 2.6869e-02, -6.6801e-03, -8.9389e-04, -3.4924e-03, -1.9667e-02,
-1.8511e-02, -7.6262e-04, -1.6382e-02, -1.5862e-02, -1.3717e-02,
 1.7528e-02, -1.1419e-03, -6.3346e-03, -1.1118e-02,  1.3159e-02,
-2.3464e-02,  2.7993e-04, -3.6273e-04,  2.3797e-02, -2.7353e-03,
-2.2223e-02,  1.3415e-02,  1.0443e-02, -2.3512e-02,  1.6832e-02,
 4.3699e-03, -1.3243e-02, -2.8605e-03,  5.4212e-03,  1.9924e-03,
-6.8664e-04, -3.9092e-04,  1.7806e-02,  1.8391e-02,  2.8473e-02,
-3.3835e-02, -1.0778e-02, -1.2371e-02, -1.9110e-03, -1.6381e-03,
```

```
 1.7288e-02, -3.9813e-03, -1.5167e-02, -1.0781e-02,  5.3808e-03,
-3.3947e-04,  3.3885e-04, -1.0162e-02, -4.0266e-03, -3.4751e-03,
 4.2359e-03, -1.4677e-03,  1.3207e-02,  7.5580e-03,  1.9397e-04,
 3.0048e-03,  8.6283e-03, -1.1193e-02,  3.8466e-02, -2.6220e-02,
-2.0251e-02, -6.3872e-03,  2.1906e-02, -7.3400e-03,  5.2753e-03,
-1.1709e-02,  8.4009e-03,  2.8530e-03, -4.7220e-03,  2.3118e-02,
-7.6039e-03,  2.8136e-03, -1.1701e-02, -4.4118e-03,  1.1846e-02,
-1.7632e-03, -1.2260e-02, -2.1210e-03,  1.2072e-02,  6.7523e-03,
-1.9128e-04, -2.5105e-02,  1.2693e-02,  1.6062e-02,  8.1264e-03,
 1.3857e-03,  3.0087e-03, -1.4111e-02,  1.9784e-02, -9.2301e-04,
-1.8428e-02,  7.8059e-03,  1.5319e-02, -1.2768e-02, -9.0166e-03,
 1.8031e-02,  2.4853e-02,  1.7788e-02,  8.8640e-03, -9.4422e-03,
-1.3652e-03,  1.2932e-02,  9.0133e-03,  1.6655e-02, -5.4321e-03,
 2.7480e-02, -3.1781e-02, -1.3331e-02,  5.5792e-03, -1.3278e-02,
-1.9219e-02, -1.3307e-02,  4.2390e-03,  3.0246e-02, -8.1990e-03,
 8.3008e-03,  1.8993e-02,  1.0643e-02,  3.1324e-02,  1.9283e-02,
 3.3642e-03,  1.9669e-02,  2.2673e-03, -1.9630e-02,  2.0147e-02,
-1.1433e-02, -7.6073e-03,  1.5071e-02, -3.0395e-03, -9.3430e-03,
-4.1657e-03,  2.2972e-03, -5.0985e-03, -1.4499e-02, -2.7673e-02,
-3.8721e-02,  5.4249e-03,  1.3504e-02, -1.2811e-03,  3.7465e-02,
 1.5154e-03,  2.4035e-02, -2.0557e-02,  9.8406e-03,  1.0352e-02,
 3.8597e-02, -1.1905e-02, -2.1718e-02,  8.3778e-03,  1.4691e-02,
 2.2631e-02, -3.7629e-03,  1.5570e-02, -9.3990e-03,  5.3536e-03,
 1.9584e-02, -1.1156e-02,  1.5190e-02,  5.4622e-03,  2.2995e-02,
 2.9260e-02, -1.5236e-03,  6.6009e-03, -3.1939e-02, -1.0486e-02,
-4.4617e-03,  3.1853e-02,  1.3736e-02,  1.3561e-02,  7.0907e-03,
-1.6753e-02, -2.5470e-02,  1.9752e-02,  2.6715e-02, -4.6859e-03,
 1.7682e-02,  3.2496e-02,  1.4553e-02,  2.6101e-02,  1.1341e-02,
-2.2271e-03,  3.5237e-02, -1.1892e-02, -1.8683e-02, -5.5245e-03,
-7.0732e-03, -5.2670e-03,  7.5946e-03, -1.8465e-02, -1.6897e-02,
 1.0127e-02,  1.3006e-02, -1.8251e-03,  6.6651e-04, -1.1207e-02,
 1.3563e-02, -1.8153e-02, -2.6487e-02,  6.0652e-03,  3.9711e-02,
-1.4285e-02,  1.8001e-02, -1.4039e-02, -1.8762e-02, -1.1778e-02,
-1.6449e-02,  9.0423e-03, -7.2730e-03,  1.7517e-02, -7.3016e-04,
 1.0212e-02,  2.3785e-02,  8.1286e-03,  8.0260e-03,  1.1922e-02,
 6.2416e-03, -2.4625e-02,  2.9461e-02, -1.4183e-02, -1.8672e-02,
-1.4057e-02,  7.9872e-03, -2.1081e-02, -2.7560e-02, -3.5690e-03,
 1.5993e-03,  9.4720e-03,  1.8272e-02, -2.3742e-03,  1.1843e-03,
-5.7722e-04,  8.7818e-03,  2.7804e-03,  6.7973e-04,  1.5877e-02,
-7.0359e-03,  2.5487e-03, -1.7925e-02,  8.4912e-03,  4.3375e-03,
 2.4508e-02,  3.6686e-03,  1.0252e-02, -1.3396e-02,  4.5706e-04,
 1.0313e-02,  1.5229e-02,  3.9907e-02, -8.0809e-03,  1.3760e-02,
-6.5863e-03,  6.6066e-03, -3.1480e-02,  2.4665e-02,  3.4374e-03,
 2.0973e-02,  1.9384e-02, -2.0880e-02,  7.1465e-03,  1.0406e-02,
 2.2273e-05, -1.9182e-02,  6.3135e-03, -1.6891e-04,  8.8664e-03,
-4.7666e-03, -1.4493e-02,  3.2176e-03,  7.3346e-03,  2.0694e-02,
-4.9972e-04,  1.8820e-02,  3.9147e-02, -2.7095e-02, -1.8293e-02,
-1.9868e-02, -9.4048e-03,  4.1552e-03,  5.3837e-03, -4.6663e-03,
-1.3019e-02, -2.4452e-02, -8.9231e-03, -1.4603e-02,  2.5529e-03,
-3.0766e-02,  1.1169e-02, -6.8113e-03, -7.5967e-03, -9.3191e-03,
 1.4919e-03, -2.3428e-03,  4.4398e-04, -1.0810e-02,  8.8498e-03,
-2.1022e-02, -8.0380e-03, -1.0818e-02, -6.4815e-03, -2.0681e-03,
 2.2326e-02, -1.9234e-02,  4.0844e-03,  7.7233e-04,  1.7226e-03,
-1.7454e-02, -1.3190e-02, -7.4112e-03, -1.7550e-03,  1.2926e-03,
-6.7029e-03, -7.0588e-03,  6.2745e-03, -1.8068e-02, -9.4855e-03,
-2.0856e-02,  8.9604e-03,  2.1294e-02,  1.7025e-02,  2.1015e-02,
```

```
 8.8233e-03, -9.8277e-03, -2.2293e-02,  2.4295e-02, -1.1174e-02,
-7.5753e-03,  6.1182e-03, -2.0653e-02, -1.6264e-02,  2.6457e-02,
-1.4782e-02,  1.8654e-02,  2.5488e-02,  2.4106e-02,  4.7888e-03,
 2.3329e-02,  3.5806e-04,  2.5154e-02,  1.7094e-02,  1.7803e-02,
 2.4687e-02,  9.0085e-03,  2.3610e-03,  2.6088e-02, -1.4110e-02,
-5.4212e-04,  8.9498e-04,  2.1150e-02,  4.8484e-03, -3.0503e-02,
-7.5025e-03, -3.3718e-02, -2.8913e-02,  1.5691e-02,  6.2047e-03,
-1.0853e-02,  1.9524e-02, -1.6188e-02,  8.9890e-03,  9.1894e-03,
-2.8592e-03, -1.0911e-02,  1.0848e-02,  4.8784e-02, -1.9687e-03,
 2.6843e-02, -4.8715e-03,  1.3489e-02, -1.4523e-02, -2.7585e-02,
 6.1228e-03,  4.8171e-03,  2.1566e-03, -3.7561e-02,  3.0775e-02,
 1.9977e-02,  1.8480e-02,  3.0368e-03,  9.3825e-03,  4.5243e-04,
 6.1650e-02, -8.6416e-03, -2.6913e-02,  6.3527e-03,  7.7985e-03,
 1.3180e-02, -1.6666e-03,  2.0865e-02,  9.9480e-03,  8.8136e-03,
 1.4841e-02,  3.3211e-03,  3.6342e-03,  2.8740e-02, -2.2120e-02,
-7.1567e-03,  1.0352e-02,  1.6433e-02,  1.1683e-02, -5.8058e-03,
-6.9297e-04,  2.6578e-02,  8.7967e-03, -3.1689e-02,  1.8949e-02,
-8.5859e-03,  3.4228e-02, -1.5237e-02, -5.9709e-03,  1.1069e-03,
-1.8394e-02, -1.9246e-02, -3.6361e-02,  3.9839e-03,  4.1237e-02,
 1.3816e-02, -7.3304e-03,  3.8832e-03,  2.4367e-03, -2.1625e-02,
-1.4523e-02, -1.6281e-04,  6.2566e-04, -1.6798e-02,  2.3083e-02,
 9.7114e-03, -8.2207e-03,  1.1595e-03, -2.0983e-02, -6.6540e-03,
-1.4097e-02,  3.4067e-03, -7.7575e-03, -1.4738e-02, -2.1343e-02,
 5.4123e-03,  3.9747e-03, -4.6185e-03, -1.5462e-02, -7.6229e-03,
 1.2211e-02, -4.8453e-03, -8.8757e-03, -1.0275e-02,  7.3482e-03,
-6.0349e-03,  2.3658e-03,  2.1053e-02, -8.5688e-03, -1.1630e-02,
-2.7332e-02, -2.0648e-02,  4.4952e-03, -1.8649e-02, -1.1564e-02,
 4.5905e-04, -6.1831e-03, -2.4435e-02, -7.1187e-03, -1.4394e-02,
-2.3544e-03,  2.1556e-02,  2.2924e-02, -1.3725e-02,  7.7785e-03,
-8.5513e-03,  2.4221e-02,  3.8192e-03,  7.0947e-04,  1.6114e-02,
 2.5932e-02,  1.8108e-02,  2.9306e-02,  1.6773e-03, -3.0166e-03,
 3.2015e-02, -1.4034e-02,  2.7365e-02, -1.8858e-02,  2.5832e-03,
 1.3498e-02, -1.3502e-02, -1.4940e-02, -1.0904e-02,  1.8642e-02,
 4.2593e-03, -1.6742e-02, -1.2638e-02, -4.5468e-02, -5.0823e-03,
-2.5093e-02,  6.7847e-03, -1.7868e-02, -7.8250e-04, -6.3448e-03],
requires_grad=True)
```

change number of features if needed

In [7]:
```
n_features = 10
model.fc = nn.Linear(model.fc.in_features, n_features)
print("model.fc: \n" , model.fc )
print("\n")
print("model: \n" , model)

# Access the first convolutional layer's weights
print("model.conv1.weight: \n:" , model.conv1.weight)

# Access the final fully connected layer's weights and biases
print("model.fc.weight: \n:" , model.fc.weight)
print("model.fc.bias: \n:" , model.fc.bias)
```

```
model.fc:
 Linear(in_features=512, out_features=10, bias=True)


model:
 ResNet(
  (conv1): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=Fal
se)
  (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=T
rue)
  (relu): ReLU(inplace=True)
  (maxpool): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=Fal
se)
  (layer1): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
    (1): BasicBlock(
      (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bia
s=False)
      (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_sta
ts=True)
    )
  )
  (layer2): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(64, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bi
as=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(64, 128, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
```

```
ias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer3): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(128, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(128, 256, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
    (1): BasicBlock(
      (conv1): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (layer4): Sequential(
    (0): BasicBlock(
      (conv1): Conv2d(256, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      )
    )
```

```
    (1): BasicBlock(
      (conv1): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
      (relu): ReLU(inplace=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), b
ias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_st
ats=True)
    )
  )
  (avgpool): AdaptiveAvgPool2d(output_size=(1, 1))
  (fc): Linear(in_features=512, out_features=10, bias=True)
)
model.conv1.weight:
: Parameter containing:
tensor([[[[-1.0419e-02, -6.1356e-03, -1.8098e-03,  ...,  5.6615e-02,
            1.7083e-02, -1.2694e-02],
          [ 1.1083e-02,  9.5276e-03, -1.0993e-01,  ..., -2.7124e-01,
           -1.2907e-01,  3.7424e-03],
          [-6.9434e-03,  5.9089e-02,  2.9548e-01,  ...,  5.1972e-01,
            2.5632e-01,  6.3573e-02],
          ...,
          [-2.7535e-02,  1.6045e-02,  7.2595e-02,  ..., -3.3285e-01,
           -4.2058e-01, -2.5781e-01],
          [ 3.0613e-02,  4.0960e-02,  6.2850e-02,  ...,  4.1384e-01,
            3.9359e-01,  1.6606e-01],
          [-1.3736e-02, -3.6746e-03, -2.4084e-02,  ..., -1.5070e-01,
           -8.2230e-02, -5.7828e-03]],

         [[-1.1397e-02, -2.6619e-02, -3.4641e-02,  ...,  3.2521e-02,
            6.6221e-04, -2.5743e-02],
          [ 4.5687e-02,  3.3603e-02, -1.0453e-01,  ..., -3.1253e-01,
           -1.6051e-01, -1.2826e-03],
          [-8.3730e-04,  9.8420e-02,  4.0210e-01,  ...,  7.0789e-01,
            3.6887e-01,  1.2455e-01],
          ...,
          [-5.5926e-02, -5.2239e-03,  2.7081e-02,  ..., -4.6178e-01,
           -5.7080e-01, -3.6552e-01],
          [ 3.2860e-02,  5.5574e-02,  9.9670e-02,  ...,  5.4636e-01,
            4.8276e-01,  1.9867e-01],
          [ 5.3051e-03,  6.6938e-03, -1.7254e-02,  ..., -1.4822e-01,
           -7.7248e-02,  7.2183e-04]],

         [[-2.0315e-03, -9.1617e-03,  2.1209e-02,  ...,  8.9177e-02,
            3.3655e-02, -2.0102e-02],
          [ 1.5398e-02, -1.8648e-02, -1.2591e-01,  ..., -2.5342e-01,
           -1.2980e-01, -2.7975e-02],
          [ 9.8454e-03,  4.9047e-02,  2.1699e-01,  ...,  3.4872e-01,
            1.0433e-01,  1.8413e-02],
          ...,
          [-2.8356e-02,  1.8404e-02,  9.8647e-02,  ..., -1.1740e-01,
           -2.5760e-01, -1.5451e-01],
          [ 2.0766e-02, -2.6286e-03, -3.7825e-02,  ...,  2.4141e-01,
            2.4345e-01,  1.1796e-01],
```

```
  [ 7.4684e-04,  7.7677e-04, -1.0050e-02,  ..., -1.4865e-01,
   -1.1754e-01, -3.8350e-02]]],


[[[-4.4154e-03, -4.0645e-03,  3.1589e-03,  ..., -3.7026e-02,
   -2.5158e-02, -4.7945e-02],
  [ 5.1310e-02,  5.3402e-02,  8.0436e-02,  ...,  1.4480e-01,
    1.4287e-01,  1.2312e-01],
  [-7.3337e-03,  2.1755e-03,  3.7580e-02,  ...,  6.1517e-02,
    8.0324e-02,  1.1715e-01],
  ...,
  [-2.6754e-02, -1.2297e-01, -1.3653e-01,  ..., -1.4068e-01,
   -1.1155e-01, -4.9556e-02],
  [ 2.3524e-02, -1.7288e-02, -1.1122e-02,  ..., -1.8826e-02,
   -2.3320e-02, -2.9474e-02],
  [ 2.8689e-02,  2.1659e-02,  4.7888e-02,  ...,  2.5498e-02,
    3.5346e-02,  1.1280e-02]],

 [[ 4.6919e-04,  1.2153e-02,  4.2035e-02,  ...,  4.6403e-02,
    4.0423e-02, -1.4439e-02],
  [ 4.3463e-02,  6.8779e-02,  1.3268e-01,  ...,  2.8606e-01,
    2.6905e-01,  2.0935e-01],
  [-5.7621e-02, -2.2642e-02,  3.0547e-02,  ...,  1.3763e-01,
    1.6538e-01,  1.7946e-01],
  ...,
  [-1.0816e-01, -2.5227e-01, -2.9742e-01,  ..., -2.8503e-01,
   -2.1493e-01, -1.0320e-01],
  [ 4.0709e-02, -3.2771e-02, -6.3450e-02,  ..., -9.2360e-02,
   -6.9876e-02, -4.9841e-02],
  [ 8.2942e-02,  8.7580e-02,  1.0111e-01,  ...,  5.2714e-02,
    6.0968e-02,  4.1198e-02]],

 [[-1.6391e-02, -1.3870e-02,  5.2810e-03,  ...,  4.3698e-02,
    2.2707e-02, -4.5983e-02],
  [ 3.3202e-02,  4.2014e-02,  9.3500e-02,  ...,  2.6162e-01,
    2.2970e-01,  1.6694e-01],
  [-4.5987e-02, -1.6365e-02,  2.6811e-02,  ...,  1.4951e-01,
    1.3216e-01,  1.3579e-01],
  ...,
  [-7.2129e-02, -1.8902e-01, -2.3389e-01,  ..., -1.9038e-01,
   -1.5609e-01, -7.5974e-02],
  [ 5.1161e-02, -2.5815e-02, -6.9357e-02,  ..., -5.8999e-02,
   -6.1550e-02, -4.4555e-02],
  [ 1.1174e-01,  7.8979e-02,  6.5849e-02,  ...,  3.1617e-02,
    2.5221e-02,  7.4257e-03]]],


[[[-7.0826e-08, -6.4306e-08, -7.3806e-08,  ..., -9.8000e-08,
   -1.0905e-07, -8.3421e-08],
  [-6.1125e-09,  2.0613e-09, -8.0922e-09,  ..., -4.9840e-08,
   -4.3836e-08, -3.0538e-09],
  [ 7.1953e-08,  7.5616e-08,  5.9282e-08,  ..., -9.7509e-09,
   -1.0951e-09,  4.2442e-08],
  ...,
  [ 9.5889e-08,  1.0039e-07,  7.9817e-08,  ..., -1.7491e-08,
   -4.7666e-08, -1.3265e-08],
```

```
  [ 1.2904e-07,  1.4762e-07,  1.7477e-07,  ...,  1.3233e-07,
    1.0628e-07,  9.3316e-08],
  [ 1.2558e-07,  1.3644e-07,  1.8431e-07,  ...,  2.1399e-07,
    1.7710e-07,  1.7166e-07]],

 [[-1.2690e-07, -9.6139e-08, -1.0372e-07,  ..., -1.1808e-07,
   -1.3309e-07, -1.0820e-07],
  [-5.7412e-08, -2.5055e-08, -3.0115e-08,  ..., -7.2922e-08,
   -6.7022e-08, -2.2574e-08],
  [ 2.1813e-08,  4.8608e-08,  3.1222e-08,  ..., -1.8694e-08,
   -7.9591e-09,  3.9750e-08],
  ...,
  [ 5.6013e-08,  7.5526e-08,  4.4496e-08,  ..., -4.4128e-08,
   -5.9930e-08, -1.8247e-08],
  [ 7.7614e-08,  9.8348e-08,  1.0455e-07,  ...,  6.3272e-08,
    4.1781e-08,  4.5901e-08],
  [ 5.9834e-08,  7.1006e-08,  9.0437e-08,  ...,  1.1654e-07,
    8.7550e-08,  9.8837e-08]],

 [[-4.3810e-08,  1.3270e-08,  7.8275e-09,  ..., -5.8804e-09,
   -2.6217e-08, -1.5649e-08],
  [ 4.1700e-08,  1.0778e-07,  1.0946e-07,  ...,  7.6403e-08,
    7.1450e-08,  9.7615e-08],
  [ 1.0436e-07,  1.6586e-07,  1.5933e-07,  ...,  1.3517e-07,
    1.3487e-07,  1.6449e-07],
  ...,
  [ 9.8763e-08,  1.5072e-07,  1.2547e-07,  ...,  6.8316e-08,
    6.8382e-08,  1.1367e-07],
  [ 9.1435e-08,  1.3576e-07,  1.3793e-07,  ...,  1.1678e-07,
    1.1723e-07,  1.4394e-07],
  [ 6.2183e-08,  8.8184e-08,  1.0456e-07,  ...,  1.3941e-07,
    1.3333e-07,  1.5844e-07]]],


...,


[[[-6.1896e-02, -3.0206e-02,  1.9225e-02,  ...,  4.3665e-02,
   -2.2114e-02, -4.2214e-02],
  [-3.8061e-02,  6.0774e-03,  4.5797e-02,  ...,  9.6029e-02,
    5.9254e-02,  2.9958e-02],
  [-2.9672e-02,  2.7766e-03,  2.0457e-02,  ...,  5.9828e-02,
    4.1422e-02,  2.3134e-02],
  ...,
  [ 1.1916e-02,  4.5701e-02,  4.4892e-02,  ...,  4.7419e-02,
    2.2274e-02, -5.4993e-03],
  [-3.2468e-02, -1.2210e-02,  2.2023e-02,  ...,  5.8061e-02,
   -7.5033e-03, -5.9736e-02],
  [-4.3314e-02, -2.8162e-02, -5.9126e-03,  ...,  8.8460e-02,
    8.4406e-03, -5.0019e-02]],

 [[-6.1292e-02, -1.4004e-02,  1.7229e-02,  ...,  1.8349e-02,
   -3.2708e-02, -4.1060e-02],
  [-3.1506e-02,  2.4460e-02,  4.5516e-02,  ...,  6.6806e-02,
    4.6687e-02,  3.3248e-02],
  [-3.2216e-02,  2.0718e-02,  2.3343e-02,  ...,  3.5265e-02,
```

```
       3.6478e-02,  3.1291e-02],
      ...,
      [ 1.7739e-02,  6.1040e-02,  4.8247e-02,  ...,  3.7785e-02,
        2.8894e-02,  1.3984e-02],
      [-1.0890e-02,  2.2079e-02,  4.2737e-02,  ...,  6.0247e-02,
        1.6197e-02, -1.2493e-02],
      [-2.2284e-02,  1.3220e-02,  3.0897e-02,  ...,  1.0403e-01,
        4.0119e-02, -5.3310e-03]],

     [[-8.5322e-02, -4.2603e-02,  6.8145e-03,  ...,  3.0751e-02,
       -3.4818e-02, -4.9945e-02],
      [-2.9215e-02,  1.8165e-02,  5.1092e-02,  ...,  9.0200e-02,
        5.3438e-02,  4.0169e-02],
      [-3.9932e-02, -1.1100e-03,  9.6176e-03,  ...,  2.4114e-02,
        2.6298e-02,  2.5489e-02],
      ...,
      [-3.1890e-03,  3.0454e-02,  1.6316e-02,  ...,  5.5054e-03,
       -6.2689e-03, -8.4638e-03],
      [-2.2995e-02, -2.8211e-03,  2.3203e-02,  ...,  3.5888e-02,
       -1.4296e-02, -3.2419e-02],
      [-9.8894e-03,  7.0542e-03,  1.0659e-02,  ...,  7.0495e-02,
        1.2996e-02, -8.3417e-03]]],


    [[[-7.8699e-03,  1.9911e-02,  3.4208e-02,  ...,  2.8694e-02,
        1.2820e-02,  1.8142e-02],
      [ 8.7942e-03, -3.2875e-02, -3.5713e-02,  ...,  7.2533e-02,
        4.5889e-02,  5.2383e-02],
      [-3.6122e-02, -1.1878e-01, -1.3767e-01,  ...,  3.3811e-02,
        3.7806e-02,  2.6944e-02],
      ...,
      [ 1.7322e-02,  3.9589e-03, -8.2269e-03,  ...,  2.7543e-03,
        1.8313e-02,  1.6057e-02],
      [-9.5007e-04,  1.6428e-02,  1.7156e-02,  ...,  3.3672e-03,
        2.2857e-02,  6.5783e-04],
      [ 6.1727e-03,  2.7145e-02,  1.4340e-02,  ...,  7.5867e-03,
        1.8770e-02,  1.5624e-02]],

     [[-1.3423e-02, -5.0696e-04,  8.0959e-03,  ..., -6.0963e-03,
        9.2341e-03,  1.5751e-02],
      [-1.8343e-02, -6.7982e-02, -7.0685e-02,  ...,  2.9855e-02,
        2.6264e-02,  2.3773e-02],
      [-5.4359e-02, -1.4663e-01, -1.6211e-01,  ...,  1.1781e-02,
        3.2477e-02,  1.1980e-02],
      ...,
      [ 8.3686e-04, -1.7564e-02, -1.9535e-02,  ..., -4.1382e-03,
        2.4658e-02,  1.2893e-02],
      [-6.3183e-04,  1.1788e-02,  2.4810e-02,  ...,  6.1105e-03,
        3.9210e-02,  9.6696e-03],
      [-7.1831e-03,  6.6918e-03,  5.2723e-03,  ..., -7.6077e-03,
        2.7253e-02,  1.7735e-02]],

     [[-2.3753e-04, -4.9343e-03,  2.2991e-03,  ..., -4.7958e-02,
       -2.6154e-02, -2.3525e-02],
      [-3.3053e-04, -5.1502e-02, -5.9977e-02,  ..., -1.7369e-02,
       -2.3337e-02, -3.7312e-02],
```

```
          [-2.2674e-02, -9.9412e-02, -1.1176e-01,  ..., -1.1725e-02,
           -8.3744e-03, -4.0615e-02],
          ...,
          [ 1.1437e-02, -8.0313e-03, -1.4955e-03,  ..., -3.4133e-02,
           -8.7267e-03, -2.3526e-02],
          [ 2.9522e-03,  6.7770e-04,  1.9933e-02,  ..., -2.2002e-02,
            1.4814e-02, -1.4487e-02],
          [-1.9085e-02, -2.9430e-02, -2.3284e-02,  ..., -4.8587e-02,
           -1.3049e-02, -2.4368e-02]]],


        [[[-3.6296e-02,  7.1996e-03,  1.9100e-02,  ...,  1.9602e-02,
            1.4870e-02, -1.7298e-02],
          [-1.1061e-02,  8.5665e-02,  1.2667e-01,  ...,  1.3744e-02,
           -5.5036e-05, -3.0162e-02],
          [ 1.1322e-01,  1.8634e-01,  5.0658e-02,  ..., -1.7333e-01,
           -7.2041e-02, -6.2474e-02],
          ...,
          [-5.3062e-02, -2.5781e-01, -2.6747e-01,  ...,  2.6781e-01,
            1.4344e-01,  5.5145e-02],
          [-2.1009e-02, -2.9969e-02,  1.0245e-01,  ...,  2.0843e-01,
           -4.1518e-03, -3.8118e-02],
          [-2.2155e-02,  1.2380e-02,  8.4302e-02,  ..., -4.4992e-02,
           -1.4687e-01, -9.0890e-02]],

         [[-5.3969e-03,  3.2799e-02,  1.5486e-02,  ..., -7.7451e-03,
            3.0229e-03,  1.1216e-03],
          [ 6.1723e-02,  1.4899e-01,  1.4645e-01,  ..., -2.8897e-02,
           -2.0227e-02, -9.1878e-03],
          [ 1.6146e-01,  2.0886e-01, -2.5589e-02,  ..., -2.7278e-01,
           -1.0735e-01, -6.2971e-02],
          ...,
          [-1.3723e-01, -4.0863e-01, -3.8551e-01,  ...,  4.0846e-01,
            2.6202e-01,  1.3491e-01],
          [-5.9388e-02, -6.1187e-02,  1.4197e-01,  ...,  3.5780e-01,
            9.0893e-02, -1.7392e-03],
          [ 7.8613e-03,  5.8403e-02,  1.5339e-01,  ...,  4.7045e-02,
           -1.0095e-01, -9.7920e-02]],

         [[-5.6799e-03,  1.3425e-02, -2.6461e-02,  ...,  4.4881e-03,
            2.0666e-03,  1.3902e-02],
          [ 6.5943e-03,  4.5181e-02,  6.0260e-02,  ...,  1.4368e-02,
           -5.0725e-03,  4.0505e-03],
          [ 5.5257e-02,  1.2397e-01,  4.3193e-02,  ..., -1.4486e-01,
           -7.4489e-02, -5.7533e-02],
          ...,
          [-3.1513e-02, -1.6334e-01, -1.5795e-01,  ...,  2.2904e-01,
            1.2017e-01,  7.1998e-02],
          [-1.0456e-02, -1.1248e-03,  8.4582e-02,  ...,  1.5748e-01,
            2.2142e-02, -1.0083e-02],
          [-4.8639e-03, -5.0065e-03,  3.6341e-02,  ..., -2.4361e-02,
           -7.1195e-02, -6.6788e-02]]]], requires_grad=True)
model.fc.weight:
: Parameter containing:
tensor([[-0.0166, -0.0325, -0.0027,  ...,  0.0415,  0.0074,  0.0252],
        [ 0.0046, -0.0348, -0.0101,  ..., -0.0329, -0.0399, -0.0235],
```

```
        [ 0.0127,  0.0429, -0.0368,  ...,  0.0294,  0.0339,  0.0413],
        ...,
        [ 0.0107,  0.0406, -0.0318,  ...,  0.0140,  0.0297, -0.0383],
        [ 0.0313,  0.0032,  0.0164,  ..., -0.0195,  0.0222,  0.0037],
        [ 0.0385, -0.0376,  0.0073,  ...,  0.0049, -0.0066,  0.0169]],
       requires_grad=True)
model.fc.bias:
: Parameter containing:
tensor([-0.0133, -0.0391, -0.0252, -0.0067, -0.0403, -0.0097,  0.0135, -0.0066,
         0.0415,  0.0095], requires_grad=True)
```

create input data

The code `torch.rand(1, 3, 64, 64)` creates a PyTorch tensor filled with random floating-point numbers sampled from a uniform distribution between 0 and 1 (specifically, on the interval ([0, 1))). (https://docs.pytorch.org/docs/stable/generated/torch.rand.html)

In machine learning and computer vision applications, this specific syntax is most commonly used to simulate a batch of image data:

- 1: The Batch Size. It generates a batch containing exactly 1 image.
- 3: The number of Channels. This represents the 3 color channels (Red, Green, Blue) of a standard RGB image.
- 64: The Height of the image in pixels.
- 64: The Width of the image in pixels.

In [23]:
```
inputData = torch.rand(1, 3, 64, 64) # imitating a 3-channel (RGB) image of resolut
print(inputData.size())
#x= torch.count_nonzero(data, dim=-2)
#print(x)
print(inputData)
```

```
torch.Size([1, 3, 64, 64])
tensor([[[[9.5841e-01, 6.0746e-01, 2.8166e-01,  ..., 9.3681e-01,
           6.1917e-01, 2.4135e-01],
          [1.6661e-01, 6.9673e-01, 6.4277e-01,  ..., 1.2155e-01,
           2.6584e-02, 1.4099e-01],
          [5.5063e-01, 5.4008e-01, 3.9730e-02,  ..., 4.8003e-01,
           6.5066e-01, 7.1473e-02],
          ...,
          [6.9182e-01, 5.7999e-01, 1.4790e-01,  ..., 8.3422e-02,
           6.3316e-01, 8.0672e-01],
          [2.4326e-01, 9.4158e-01, 6.8585e-01,  ..., 7.0906e-01,
           7.0271e-01, 3.0782e-01],
          [3.3045e-01, 7.6311e-01, 2.2229e-01,  ..., 6.8798e-01,
           5.2555e-01, 8.8321e-01]],

         [[6.7163e-01, 7.9909e-01, 5.3082e-01,  ..., 6.0547e-01,
           8.3251e-01, 7.6736e-01],
          [9.3180e-01, 7.0346e-02, 3.9334e-01,  ..., 3.4731e-01,
           3.2179e-01, 1.4288e-02],
          [8.0640e-01, 7.2000e-01, 8.1161e-01,  ..., 5.3906e-01,
           7.1333e-01, 6.7825e-01],
          ...,
          [2.6193e-01, 6.2501e-02, 1.8817e-01,  ..., 3.4501e-02,
           9.5582e-01, 4.5406e-01],
          [2.2799e-01, 3.9165e-01, 1.4729e-02,  ..., 2.2498e-01,
           4.5451e-01, 4.1192e-01],
          [9.8269e-01, 7.4705e-01, 8.5231e-01,  ..., 7.1067e-01,
           2.8151e-01, 8.7332e-01]],

         [[4.8994e-01, 8.7905e-01, 5.8813e-01,  ..., 7.6099e-01,
           1.5596e-01, 3.4634e-01],
          [4.3577e-01, 7.1509e-01, 5.1868e-01,  ..., 1.3920e-01,
           1.3287e-01, 8.8131e-01],
          [2.7214e-03, 6.3753e-01, 6.1534e-01,  ..., 8.2451e-01,
           2.7781e-01, 1.8892e-01],
          ...,
          [9.3951e-01, 7.3669e-02, 4.5974e-01,  ..., 8.0792e-01,
           1.8557e-01, 4.8001e-01],
          [9.7458e-02, 9.1575e-01, 4.9707e-01,  ..., 7.7584e-01,
           7.4184e-01, 2.2200e-01],
          [6.3333e-01, 8.4113e-01, 8.1350e-01,  ..., 9.8211e-04,
           6.7820e-01, 4.8692e-01]]]])
```

Visualize the data: functions to show an image

In [9]:
```
import torch
import matplotlib.pyplot as plt
import numpy as np
import torchvision

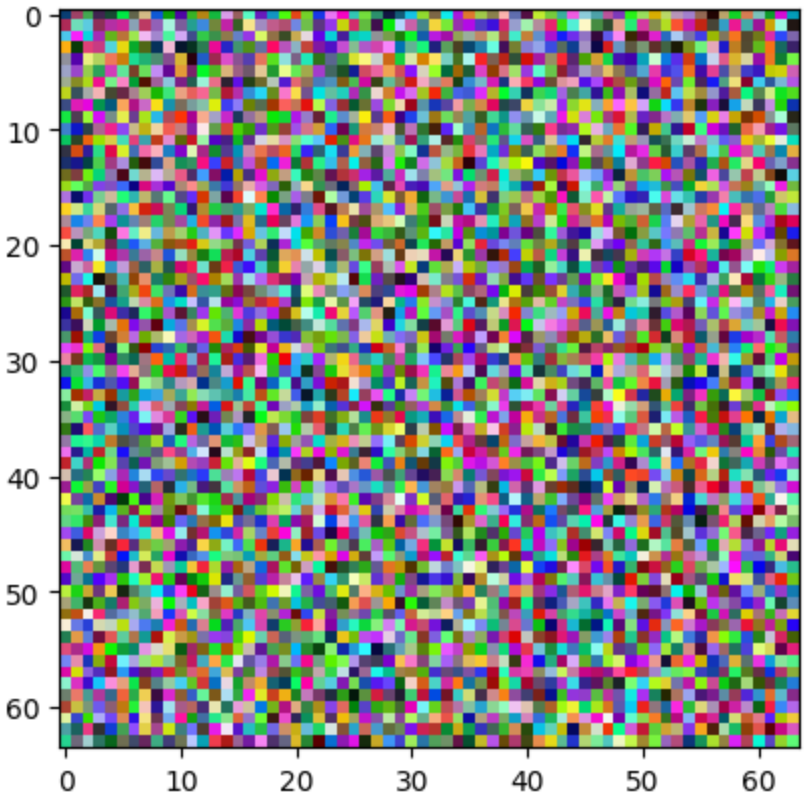
# 1. Define the original function (without unnormalization for raw random data)
def Display(img):
    print("img shape:", img.shape)  # Helpful debugging check
    npimg = img.numpy()
    # Transpose from (Channels, Height, Width) to (Height, Width, Channels)
```

```
    plt.imshow(np.transpose(npimg, (1, 2, 0)))
    plt.show()


# 3. Create a grid (make_grid converts a 4D batch into a single 3D image grid)
image = torchvision.utils.make_grid(inputData)

# 4. Display the grid image
Display(image)
```

```
img shape: torch.Size([3, 64, 64])
```



create labels

- random in this demo
- the **size of the labels should be compatible with features** (and therefore with predictions)

In [24]:
```
# Generates a matrix of shape (1, n_features) filled with random floating-point val
labels = torch.rand(1, n_features)  #Because these are random floats, the probabili
print("labels:",labels)
print(labels.size())

# count how many elements along that row are not equal to 0. Since torch.rand yield
#x= torch.count_nonzero(labels, dim=1) #
#print(x)
```

```
labels: tensor([[0.3839, 0.1817, 0.1820, 0.0781, 0.6000, 0.2717, 0.2384, 0.4251, 0.4
012,
         0.1795]])
torch.Size([1, 10])
```

Next, we run the input data through the model through each of its layers to make a prediction. This is the *forward pass*.

In [25]:
```
prediction = model(inputData) # forward pass
print("prediction: \n", prediction)
#x= torch.count_nonzero(prediction, dim=+1)
print(prediction.size())
```

```
prediction:
 tensor([[-0.8056, -1.1892, -0.9612,  0.8434,  0.0793, -0.3005, -0.6426, -0.5022,
         -0.5883,  0.9827]], grad_fn=<AddmmBackward0>)
torch.Size([1, 10])
```

We use the model's prediction and the corresponding label to calculate the error (*loss*).

In [26]:
```
loss = (prediction - labels).sum()
print("loss; \n" , loss)
```

```
loss;
 tensor(-6.0259, grad_fn=<SumBackward0>)
```

The next step is to backpropagate this error through the network. Backward propagation is kicked off when we call `.backward()` on the error tensor. Autograd then calculates and stores the gradients for each model parameter in the parameter's `.grad` attribute.

In [27]:
```
loss.backward() # backward pass
```

Next, we load an optimizer, in this case stochastic gradient descent **SGD with a learning rate of 0.01 and momentum of 0.9. We register all the parameters of the model in the optimizer.**

In [29]:
```
optim = torch.optim.SGD(model.parameters(), lr=1e-2, momentum=0.9)
```

Finally, we call `.step()` to initiate gradient descent. The optimizer adjusts each parameter by its gradient stored in `.grad`.

In [30]:
```
optim.step() #gradient descent

# Access the first convolutional layer's weights
print("model.conv1.weight: \n:" , model.conv1.weight)

# Access the final fully connected layer's weights and biases
print("model.fc.weight: \n:" , model.fc.weight)
print("model.fc.bias: \n:" , model.fc.bias)
```

```
model.conv1.weight:
: Parameter containing:
tensor([[[[-1.0419e-02, -6.1356e-03, -1.8098e-03,  ...,  5.6615e-02,
            1.7083e-02, -1.2694e-02],
          [ 1.1083e-02,  9.5276e-03, -1.0993e-01,  ..., -2.7124e-01,
           -1.2907e-01,  3.7424e-03],
          [-6.9434e-03,  5.9089e-02,  2.9548e-01,  ...,  5.1972e-01,
            2.5632e-01,  6.3573e-02],
          ...,
          [-2.7535e-02,  1.6045e-02,  7.2595e-02,  ..., -3.3285e-01,
           -4.2058e-01, -2.5781e-01],
          [ 3.0613e-02,  4.0960e-02,  6.2850e-02,  ...,  4.1384e-01,
            3.9359e-01,  1.6606e-01],
          [-1.3736e-02, -3.6746e-03, -2.4084e-02,  ..., -1.5070e-01,
           -8.2230e-02, -5.7828e-03]],

         [[-1.1397e-02, -2.6619e-02, -3.4641e-02,  ...,  3.2521e-02,
            6.6221e-04, -2.5743e-02],
          [ 4.5687e-02,  3.3603e-02, -1.0453e-01,  ..., -3.1253e-01,
           -1.6051e-01, -1.2826e-03],
          [-8.3730e-04,  9.8420e-02,  4.0210e-01,  ...,  7.0789e-01,
            3.6887e-01,  1.2455e-01],
          ...,
          [-5.5926e-02, -5.2239e-03,  2.7081e-02,  ..., -4.6178e-01,
           -5.7080e-01, -3.6552e-01],
          [ 3.2860e-02,  5.5574e-02,  9.9670e-02,  ...,  5.4636e-01,
            4.8276e-01,  1.9867e-01],
          [ 5.3051e-03,  6.6938e-03, -1.7254e-02,  ..., -1.4822e-01,
           -7.7248e-02,  7.2183e-04]],

         [[-2.0315e-03, -9.1617e-03,  2.1209e-02,  ...,  8.9177e-02,
            3.3655e-02, -2.0102e-02],
          [ 1.5398e-02, -1.8648e-02, -1.2591e-01,  ..., -2.5342e-01,
           -1.2980e-01, -2.7975e-02],
          [ 9.8454e-03,  4.9047e-02,  2.1699e-01,  ...,  3.4872e-01,
            1.0433e-01,  1.8413e-02],
          ...,
          [-2.8356e-02,  1.8404e-02,  9.8647e-02,  ..., -1.1740e-01,
           -2.5760e-01, -1.5451e-01],
          [ 2.0766e-02, -2.6286e-03, -3.7825e-02,  ...,  2.4141e-01,
            2.4345e-01,  1.1796e-01],
          [ 7.4684e-04,  7.7677e-04, -1.0050e-02,  ..., -1.4865e-01,
           -1.1754e-01, -3.8350e-02]]],


        [[[-4.4154e-03, -4.0645e-03,  3.1589e-03,  ..., -3.7026e-02,
           -2.5158e-02, -4.7945e-02],
          [ 5.1310e-02,  5.3402e-02,  8.0436e-02,  ...,  1.4480e-01,
            1.4287e-01,  1.2312e-01],
          [-7.3337e-03,  2.1755e-03,  3.7580e-02,  ...,  6.1517e-02,
            8.0324e-02,  1.1715e-01],
          ...,
          [-2.6754e-02, -1.2297e-01, -1.3653e-01,  ..., -1.4068e-01,
           -1.1155e-01, -4.9556e-02],
          [ 2.3524e-02, -1.7288e-02, -1.1122e-02,  ..., -1.8826e-02,
           -2.3320e-02, -2.9474e-02],
```

```
 [ 2.8689e-02,  2.1659e-02,  4.7888e-02,  ...,  2.5498e-02,
   3.5346e-02,  1.1280e-02]],

[[ 4.6919e-04,  1.2153e-02,  4.2035e-02,  ...,  4.6403e-02,
   4.0423e-02, -1.4439e-02],
 [ 4.3463e-02,  6.8779e-02,  1.3268e-01,  ...,  2.8606e-01,
   2.6905e-01,  2.0935e-01],
 [-5.7621e-02, -2.2642e-02,  3.0547e-02,  ...,  1.3763e-01,
   1.6538e-01,  1.7946e-01],
 ...,
 [-1.0816e-01, -2.5227e-01, -2.9742e-01,  ..., -2.8503e-01,
  -2.1493e-01, -1.0320e-01],
 [ 4.0709e-02, -3.2771e-02, -6.3450e-02,  ..., -9.2360e-02,
  -6.9876e-02, -4.9841e-02],
 [ 8.2942e-02,  8.7580e-02,  1.0111e-01,  ...,  5.2714e-02,
   6.0968e-02,  4.1198e-02]],

[[-1.6391e-02, -1.3870e-02,  5.2810e-03,  ...,  4.3698e-02,
   2.2707e-02, -4.5983e-02],
 [ 3.3202e-02,  4.2014e-02,  9.3500e-02,  ...,  2.6162e-01,
   2.2970e-01,  1.6694e-01],
 [-4.5987e-02, -1.6365e-02,  2.6811e-02,  ...,  1.4951e-01,
   1.3216e-01,  1.3579e-01],
 ...,
 [-7.2129e-02, -1.8902e-01, -2.3389e-01,  ..., -1.9038e-01,
  -1.5609e-01, -7.5974e-02],
 [ 5.1161e-02, -2.5815e-02, -6.9357e-02,  ..., -5.8999e-02,
  -6.1550e-02, -4.4555e-02],
 [ 1.1174e-01,  7.8979e-02,  6.5849e-02,  ...,  3.1617e-02,
   2.5221e-02,  7.4257e-03]]],


[[[-7.0826e-08, -6.4306e-08, -7.3806e-08,  ..., -9.8000e-08,
   -1.0905e-07, -8.3421e-08],
  [-6.1125e-09,  2.0613e-09, -8.0922e-09,  ..., -4.9840e-08,
   -4.3836e-08, -3.0538e-09],
  [ 7.1953e-08,  7.5616e-08,  5.9282e-08,  ..., -9.7509e-09,
   -1.0951e-09,  4.2442e-08],
  ...,
  [ 9.5889e-08,  1.0039e-07,  7.9817e-08,  ..., -1.7491e-08,
   -4.7666e-08, -1.3265e-08],
  [ 1.2904e-07,  1.4762e-07,  1.7477e-07,  ...,  1.3233e-07,
    1.0628e-07,  9.3316e-08],
  [ 1.2558e-07,  1.3644e-07,  1.8431e-07,  ...,  2.1399e-07,
    1.7710e-07,  1.7166e-07]],

 [[-1.2690e-07, -9.6139e-08, -1.0372e-07,  ..., -1.1808e-07,
   -1.3309e-07, -1.0820e-07],
  [-5.7412e-08, -2.5055e-08, -3.0115e-08,  ..., -7.2922e-08,
   -6.7022e-08, -2.2574e-08],
  [ 2.1813e-08,  4.8608e-08,  3.1222e-08,  ..., -1.8694e-08,
   -7.9591e-09,  3.9750e-08],
  ...,
  [ 5.6013e-08,  7.5526e-08,  4.4496e-08,  ..., -4.4128e-08,
   -5.9930e-08, -1.8247e-08],
  [ 7.7614e-08,  9.8348e-08,  1.0455e-07,  ...,  6.3272e-08,
```

```
    4.1781e-08,  4.5901e-08],
   [ 5.9834e-08,  7.1006e-08,  9.0437e-08,  ...,  1.1654e-07,
     8.7550e-08,  9.8837e-08]],

  [[-4.3810e-08,  1.3270e-08,  7.8275e-09,  ..., -5.8804e-09,
    -2.6217e-08, -1.5649e-08],
   [ 4.1700e-08,  1.0778e-07,  1.0946e-07,  ...,  7.6403e-08,
     7.1450e-08,  9.7615e-08],
   [ 1.0436e-07,  1.6586e-07,  1.5933e-07,  ...,  1.3517e-07,
     1.3487e-07,  1.6449e-07],
   ...,
   [ 9.8763e-08,  1.5072e-07,  1.2547e-07,  ...,  6.8316e-08,
     6.8382e-08,  1.1367e-07],
   [ 9.1435e-08,  1.3576e-07,  1.3793e-07,  ...,  1.1678e-07,
     1.1723e-07,  1.4394e-07],
   [ 6.2183e-08,  8.8184e-08,  1.0456e-07,  ...,  1.3941e-07,
     1.3333e-07,  1.5844e-07]]],


...,


[[[-6.1896e-02, -3.0206e-02,  1.9225e-02,  ...,  4.3665e-02,
    -2.2114e-02, -4.2214e-02],
   [-3.8061e-02,  6.0774e-03,  4.5797e-02,  ...,  9.6029e-02,
     5.9254e-02,  2.9958e-02],
   [-2.9672e-02,  2.7766e-03,  2.0457e-02,  ...,  5.9828e-02,
     4.1422e-02,  2.3134e-02],
   ...,
   [ 1.1916e-02,  4.5701e-02,  4.4892e-02,  ...,  4.7419e-02,
     2.2274e-02, -5.4993e-03],
   [-3.2468e-02, -1.2210e-02,  2.2023e-02,  ...,  5.8061e-02,
    -7.5033e-03, -5.9736e-02],
   [-4.3314e-02, -2.8162e-02, -5.9126e-03,  ...,  8.8460e-02,
     8.4406e-03, -5.0019e-02]],

  [[-6.1292e-02, -1.4004e-02,  1.7229e-02,  ...,  1.8349e-02,
    -3.2708e-02, -4.1060e-02],
   [-3.1506e-02,  2.4460e-02,  4.5516e-02,  ...,  6.6806e-02,
     4.6687e-02,  3.3248e-02],
   [-3.2216e-02,  2.0718e-02,  2.3343e-02,  ...,  3.5265e-02,
     3.6478e-02,  3.1291e-02],
   ...,
   [ 1.7739e-02,  6.1040e-02,  4.8247e-02,  ...,  3.7785e-02,
     2.8894e-02,  1.3984e-02],
   [-1.0890e-02,  2.2079e-02,  4.2737e-02,  ...,  6.0247e-02,
     1.6197e-02, -1.2493e-02],
   [-2.2284e-02,  1.3220e-02,  3.0897e-02,  ...,  1.0403e-01,
     4.0119e-02, -5.3310e-03]],

  [[-8.5322e-02, -4.2603e-02,  6.8145e-03,  ...,  3.0751e-02,
    -3.4818e-02, -4.9945e-02],
   [-2.9215e-02,  1.8165e-02,  5.1092e-02,  ...,  9.0200e-02,
     5.3438e-02,  4.0169e-02],
   [-3.9932e-02, -1.1100e-03,  9.6176e-03,  ...,  2.4114e-02,
     2.6298e-02,  2.5489e-02],
```

```
        ...,
        [-3.1890e-03,  3.0454e-02,  1.6316e-02,  ...,  5.5054e-03,
         -6.2689e-03, -8.4638e-03],
        [-2.2995e-02, -2.8211e-03,  2.3203e-02,  ...,  3.5888e-02,
         -1.4296e-02, -3.2419e-02],
        [-9.8894e-03,  7.0542e-03,  1.0659e-02,  ...,  7.0495e-02,
          1.2996e-02, -8.3417e-03]]],


      [[[-7.8699e-03,  1.9911e-02,  3.4208e-02,  ...,  2.8694e-02,
          1.2820e-02,  1.8142e-02],
        [ 8.7942e-03, -3.2875e-02, -3.5713e-02,  ...,  7.2533e-02,
          4.5889e-02,  5.2383e-02],
        [-3.6122e-02, -1.1878e-01, -1.3767e-01,  ...,  3.3811e-02,
          3.7806e-02,  2.6944e-02],
        ...,
        [ 1.7322e-02,  3.9589e-03, -8.2269e-03,  ...,  2.7543e-03,
          1.8313e-02,  1.6057e-02],
        [-9.5007e-04,  1.6428e-02,  1.7156e-02,  ...,  3.3672e-03,
          2.2857e-02,  6.5783e-04],
        [ 6.1727e-03,  2.7145e-02,  1.4340e-02,  ...,  7.5867e-03,
          1.8770e-02,  1.5624e-02]],

       [[-1.3423e-02, -5.0696e-04,  8.0959e-03,  ..., -6.0963e-03,
          9.2341e-03,  1.5751e-02],
        [-1.8343e-02, -6.7982e-02, -7.0685e-02,  ...,  2.9855e-02,
          2.6264e-02,  2.3773e-02],
        [-5.4359e-02, -1.4663e-01, -1.6211e-01,  ...,  1.1781e-02,
          3.2477e-02,  1.1980e-02],
        ...,
        [ 8.3686e-04, -1.7564e-02, -1.9535e-02,  ..., -4.1382e-03,
          2.4658e-02,  1.2893e-02],
        [-6.3183e-04,  1.1788e-02,  2.4810e-02,  ...,  6.1105e-03,
          3.9210e-02,  9.6696e-03],
        [-7.1831e-03,  6.6918e-03,  5.2723e-03,  ..., -7.6077e-03,
          2.7253e-02,  1.7735e-02]],

       [[-2.3753e-04, -4.9343e-03,  2.2991e-03,  ..., -4.7958e-02,
         -2.6154e-02, -2.3525e-02],
        [-3.3053e-04, -5.1502e-02, -5.9977e-02,  ..., -1.7369e-02,
         -2.3337e-02, -3.7312e-02],
        [-2.2674e-02, -9.9412e-02, -1.1176e-01,  ..., -1.1725e-02,
         -8.3744e-03, -4.0615e-02],
        ...,
        [ 1.1437e-02, -8.0313e-03, -1.4955e-03,  ..., -3.4133e-02,
         -8.7267e-03, -2.3526e-02],
        [ 2.9522e-03,  6.7770e-04,  1.9933e-02,  ..., -2.2002e-02,
          1.4814e-02, -1.4487e-02],
        [-1.9085e-02, -2.9430e-02, -2.3284e-02,  ..., -4.8587e-02,
         -1.3049e-02, -2.4368e-02]]],


      [[[-3.6296e-02,  7.1996e-03,  1.9100e-02,  ...,  1.9602e-02,
          1.4870e-02, -1.7298e-02],
        [-1.1061e-02,  8.5665e-02,  1.2667e-01,  ...,  1.3744e-02,
         -5.5036e-05, -3.0162e-02],
```

```
           [ 1.1322e-01,  1.8634e-01,  5.0658e-02,  ..., -1.7333e-01,
            -7.2041e-02, -6.2474e-02],
           ...,
           [-5.3062e-02, -2.5781e-01, -2.6747e-01,  ...,  2.6781e-01,
             1.4344e-01,  5.5145e-02],
           [-2.1009e-02, -2.9969e-02,  1.0245e-01,  ...,  2.0843e-01,
            -4.1518e-03, -3.8118e-02],
           [-2.2155e-02,  1.2380e-02,  8.4302e-02,  ..., -4.4992e-02,
            -1.4687e-01, -9.0890e-02]],

          [[-5.3969e-03,  3.2799e-02,  1.5486e-02,  ..., -7.7451e-03,
             3.0229e-03,  1.1216e-03],
           [ 6.1723e-02,  1.4899e-01,  1.4645e-01,  ..., -2.8897e-02,
            -2.0227e-02, -9.1878e-03],
           [ 1.6146e-01,  2.0886e-01, -2.5589e-02,  ..., -2.7278e-01,
            -1.0735e-01, -6.2971e-02],
           ...,
           [-1.3723e-01, -4.0863e-01, -3.8551e-01,  ...,  4.0846e-01,
             2.6202e-01,  1.3491e-01],
           [-5.9388e-02, -6.1187e-02,  1.4197e-01,  ...,  3.5780e-01,
             9.0893e-02, -1.7392e-03],
           [ 7.8613e-03,  5.8403e-02,  1.5339e-01,  ...,  4.7045e-02,
            -1.0095e-01, -9.7920e-02]],

          [[-5.6799e-03,  1.3425e-02, -2.6461e-02,  ...,  4.4881e-03,
             2.0666e-03,  1.3902e-02],
           [ 6.5943e-03,  4.5181e-02,  6.0260e-02,  ...,  1.4368e-02,
            -5.0725e-03,  4.0505e-03],
           [ 5.5257e-02,  1.2397e-01,  4.3193e-02,  ..., -1.4486e-01,
            -7.4489e-02, -5.7533e-02],
           ...,
           [-3.1513e-02, -1.6334e-01, -1.5795e-01,  ...,  2.2904e-01,
             1.2017e-01,  7.1998e-02],
           [-1.0456e-02, -1.1248e-03,  8.4582e-02,  ...,  1.5748e-01,
             2.2142e-02, -1.0083e-02],
           [-4.8639e-03, -5.0065e-03,  3.6341e-02,  ..., -2.4361e-02,
            -7.1195e-02, -6.6788e-02]]])
model.fc.weight:
: Parameter containing:
tensor([[ 0.0239,  0.0007,  0.0160,  ..., -0.0038, -0.0065, -0.0493],
        [-0.0181, -0.0027, -0.0182,  ...,  0.0116, -0.0535, -0.0410],
        [ 0.0030, -0.0332, -0.0371,  ...,  0.0209, -0.0114, -0.0002],
        ...,
        [ 0.0300, -0.0078, -0.0450,  ...,  0.0222, -0.0317, -0.0387],
        [-0.0316,  0.0289,  0.0025,  ...,  0.0335, -0.0127, -0.0493],
        [-0.0281,  0.0035, -0.0534,  ..., -0.0429, -0.0104, -0.0041]],
       requires_grad=True)
model.fc.bias:
: Parameter containing:
tensor([-0.0447,  0.0140,  0.0217, -0.0294, -0.0181, -0.0119,  0.0264,  0.0185,
        -0.0064,  0.0130], requires_grad=True)
```

Differentiation in Autograd

Let's take a look at how `autograd` collects gradients. We create two tensors `a` and `b` with `requires_grad=True`. This signals to `autograd` that every operation on them should be tracked.

```
import torch

a = torch.tensor([2., 3.], requires_grad=True)
b = torch.tensor([6., 4.], requires_grad=True)
print(a)
print(b)
```

```
tensor([2., 3.], requires_grad=True)
tensor([6., 4.], requires_grad=True)
```

We create another tensor `Q` from `a` and `b`.



```
Q = 3*a**3 - b**2
```

Let's assume `a` and `b` to be parameters of an NN, and `Q` to be the error. In NN training, we want gradients of the error w.r.t. parameters, i.e.

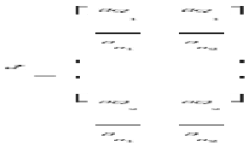


When we call `.backward()` on `Q`, autograd calculates these gradients and stores them in the respective tensors' `.grad` attribute.

We need to explicitly pass a `gradient` argument in `Q.backward()` because it is a vector. `gradient` is a tensor of the same shape as `Q`, and it represents the gradient of Q w.r.t. itself, i.e.



Mathematically, the relationship between and forms a Jacobian matrix :



```
external_grad = torch.tensor([1., 1.])

Q.backward(gradient=external_grad)
```

Gradients are now deposited in `a.grad` and

check if collected gradients are correct

In [19]:
```
print("a.grad:", a.grad)
print("b.grad:", b.grad)
print(9*a**2 == a.grad)
print(-2*b == b.grad)
```

```
a.grad: tensor([36., 81.])
b.grad: tensor([-12.,  -8.])
tensor([True, True])
tensor([True, True])
```

Computational Graph

Conceptually, autograd keeps a record of data (tensors) & all executed operations (along with the resulting new tensors) in a directed acyclic graph (DAG) consisting of [Function] (https://pytorch.org/docs/stable/autograd.html#torch.autograd.Function) objects. In this DAG, leaves are the input tensors, roots are the output tensors. By tracing this graph from roots to leaves, you can automatically compute the gradients using the chain rule.

In a forward pass, autograd does two things simultaneously:

- run the requested operation to compute a resulting tensor, and
- maintain the operation's *gradient function* in the DAG.

The backward pass kicks off when `.backward()` is called on the DAG root. `autograd` then:

- computes the gradients from each `.grad_fn`,
- accumulates them in the respective tensor's `.grad` attribute, and
- using the chain rule, propagates all the way to the leaf tensors.**

** NOTE: DAGs are dynamic in PyTorch** An important thing to note is that the graph is recreated from scratch; after each `.backward()` call, autograd starts populating a new graph. This is exactly what allows you to use control flow statements in your model; you can change the shape, size and operations at every iteration if needed.

In [3]:
```
x = torch.rand(5, 5)
y = torch.rand(5, 5)
z = torch.rand((5, 5), requires_grad=True)

a = x + y
print(f"Does `a` require gradients? : {a.requires_grad}")
b = x + z
print(f"Does `b` require gradients?: {b.requires_grad}")
```

```
Does `a` require gradients? : False
Does `b` require gradients?: True
```

Freezing layers and transfer learning

In a NN, parameters that don't compute gradients are usually called **frozen parameters. It is useful to "freeze" part of your model if you know in advance that you won't need the gradients of those parameters (this offers some performance benefits by reducing autograd computations).**

Another common usecase where exclusion from the DAG is important is for transfer learning (see finetuning a pretrained network)

In transfer learning, we freeze most of the model and typically only modify the classifier layers (last fully connected layers) to make predictions on new labels. Let's walk through a small example to demonstrate this. As before, we load a pretrained resnet18 model, and freeze all the parameters.

In [20]:
```
from torch import nn, optim

#model = torchvision.models.resnet18(pretrained=True)
model = resnet18(weights=None)

PATH = r'D:\Teaching\UL\AI\Lab\resnetModel\resnet18.pth'
#LOAD model parameters in dictionary
model.load_state_dict(torch.load(PATH))

#print(model)
```

```
print("model.conv1:\n", model.conv1 )
print("model.conv1.weight:\n", model.conv1.weight)
print("++++++++++++++++++++++++++++++++++++")
print("model.fc:\n", model.fc )
print("model.fc.weight:\n", model.fc.weight)

# Freeze all the parameters in the network
# Iterate through every parameter (weights and biases) in the model
for param in model.parameters():
    # Disable gradient tracking for the parameter to freeze it,
    # reducing memory usage and preventing parameter updates during backward()
    param.requires_grad = False

print("+++++++++++++++++ After requires_grad = False ++++++++++++++++++++")

print("model.conv1:\n", model.conv1 )
print("model.conv1.weight:\n", model.conv1.weight)
print("++++++++++++++++++++++++++++++++++++")
print("model.fc:\n", model.fc )
print("model.fc.weight:\n", model.fc.weight)
```

```
model.conv1:
 Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
model.conv1.weight:
 Parameter containing:
tensor([[[[-1.0419e-02, -6.1356e-03, -1.8098e-03,  ...,  5.6615e-02,
            1.7083e-02, -1.2694e-02],
          [ 1.1083e-02,  9.5276e-03, -1.0993e-01,  ..., -2.7124e-01,
           -1.2907e-01,  3.7424e-03],
          [-6.9434e-03,  5.9089e-02,  2.9548e-01,  ...,  5.1972e-01,
            2.5632e-01,  6.3573e-02],
          ...,
          [-2.7535e-02,  1.6045e-02,  7.2595e-02,  ..., -3.3285e-01,
           -4.2058e-01, -2.5781e-01],
          [ 3.0613e-02,  4.0960e-02,  6.2850e-02,  ...,  4.1384e-01,
            3.9359e-01,  1.6606e-01],
          [-1.3736e-02, -3.6746e-03, -2.4084e-02,  ..., -1.5070e-01,
           -8.2230e-02, -5.7828e-03]],

         [[-1.1397e-02, -2.6619e-02, -3.4641e-02,  ...,  3.2521e-02,
            6.6221e-04, -2.5743e-02],
          [ 4.5687e-02,  3.3603e-02, -1.0453e-01,  ..., -3.1253e-01,
           -1.6051e-01, -1.2826e-03],
          [-8.3730e-04,  9.8420e-02,  4.0210e-01,  ...,  7.0789e-01,
            3.6887e-01,  1.2455e-01],
          ...,
          [-5.5926e-02, -5.2239e-03,  2.7081e-02,  ..., -4.6178e-01,
           -5.7080e-01, -3.6552e-01],
          [ 3.2860e-02,  5.5574e-02,  9.9670e-02,  ...,  5.4636e-01,
            4.8276e-01,  1.9867e-01],
          [ 5.3051e-03,  6.6938e-03, -1.7254e-02,  ..., -1.4822e-01,
           -7.7248e-02,  7.2183e-04]],

         [[-2.0315e-03, -9.1617e-03,  2.1209e-02,  ...,  8.9177e-02,
            3.3655e-02, -2.0102e-02],
          [ 1.5398e-02, -1.8648e-02, -1.2591e-01,  ..., -2.5342e-01,
           -1.2980e-01, -2.7975e-02],
          [ 9.8454e-03,  4.9047e-02,  2.1699e-01,  ...,  3.4872e-01,
            1.0433e-01,  1.8413e-02],
          ...,
          [-2.8356e-02,  1.8404e-02,  9.8647e-02,  ..., -1.1740e-01,
           -2.5760e-01, -1.5451e-01],
          [ 2.0766e-02, -2.6286e-03, -3.7825e-02,  ...,  2.4141e-01,
            2.4345e-01,  1.1796e-01],
          [ 7.4684e-04,  7.7677e-04, -1.0050e-02,  ..., -1.4865e-01,
           -1.1754e-01, -3.8350e-02]]],


        [[[-4.4154e-03, -4.0645e-03,  3.1589e-03,  ..., -3.7026e-02,
           -2.5158e-02, -4.7945e-02],
          [ 5.1310e-02,  5.3402e-02,  8.0436e-02,  ...,  1.4480e-01,
            1.4287e-01,  1.2312e-01],
          [-7.3337e-03,  2.1755e-03,  3.7580e-02,  ...,  6.1517e-02,
            8.0324e-02,  1.1715e-01],
          ...,
          [-2.6754e-02, -1.2297e-01, -1.3653e-01,  ..., -1.4068e-01,
           -1.1155e-01, -4.9556e-02],
```

```
 [ 2.3524e-02, -1.7288e-02, -1.1122e-02,  ..., -1.8826e-02,
  -2.3320e-02, -2.9474e-02],
 [ 2.8689e-02,  2.1659e-02,  4.7888e-02,  ...,  2.5498e-02,
   3.5346e-02,  1.1280e-02]],

[[ 4.6919e-04,  1.2153e-02,  4.2035e-02,  ...,  4.6403e-02,
   4.0423e-02, -1.4439e-02],
 [ 4.3463e-02,  6.8779e-02,  1.3268e-01,  ...,  2.8606e-01,
   2.6905e-01,  2.0935e-01],
 [-5.7621e-02, -2.2642e-02,  3.0547e-02,  ...,  1.3763e-01,
   1.6538e-01,  1.7946e-01],
 ...,
 [-1.0816e-01, -2.5227e-01, -2.9742e-01,  ..., -2.8503e-01,
  -2.1493e-01, -1.0320e-01],
 [ 4.0709e-02, -3.2771e-02, -6.3450e-02,  ..., -9.2360e-02,
  -6.9876e-02, -4.9841e-02],
 [ 8.2942e-02,  8.7580e-02,  1.0111e-01,  ...,  5.2714e-02,
   6.0968e-02,  4.1198e-02]],

[[-1.6391e-02, -1.3870e-02,  5.2810e-03,  ...,  4.3698e-02,
   2.2707e-02, -4.5983e-02],
 [ 3.3202e-02,  4.2014e-02,  9.3500e-02,  ...,  2.6162e-01,
   2.2970e-01,  1.6694e-01],
 [-4.5987e-02, -1.6365e-02,  2.6811e-02,  ...,  1.4951e-01,
   1.3216e-01,  1.3579e-01],
 ...,
 [-7.2129e-02, -1.8902e-01, -2.3389e-01,  ..., -1.9038e-01,
  -1.5609e-01, -7.5974e-02],
 [ 5.1161e-02, -2.5815e-02, -6.9357e-02,  ..., -5.8999e-02,
  -6.1550e-02, -4.4555e-02],
 [ 1.1174e-01,  7.8979e-02,  6.5849e-02,  ...,  3.1617e-02,
   2.5221e-02,  7.4257e-03]]],


[[[-7.0826e-08, -6.4306e-08, -7.3806e-08,  ..., -9.8000e-08,
  -1.0905e-07, -8.3421e-08],
 [-6.1125e-09,  2.0613e-09, -8.0922e-09,  ..., -4.9840e-08,
  -4.3836e-08, -3.0538e-09],
 [ 7.1953e-08,  7.5616e-08,  5.9282e-08,  ..., -9.7509e-09,
  -1.0951e-09,  4.2442e-08],
 ...,
 [ 9.5889e-08,  1.0039e-07,  7.9817e-08,  ..., -1.7491e-08,
  -4.7666e-08, -1.3265e-08],
 [ 1.2904e-07,  1.4762e-07,  1.7477e-07,  ...,  1.3233e-07,
   1.0628e-07,  9.3316e-08],
 [ 1.2558e-07,  1.3644e-07,  1.8431e-07,  ...,  2.1399e-07,
   1.7710e-07,  1.7166e-07]],

[[-1.2690e-07, -9.6139e-08, -1.0372e-07,  ..., -1.1808e-07,
  -1.3309e-07, -1.0820e-07],
 [-5.7412e-08, -2.5055e-08, -3.0115e-08,  ..., -7.2922e-08,
  -6.7022e-08, -2.2574e-08],
 [ 2.1813e-08,  4.8608e-08,  3.1222e-08,  ..., -1.8694e-08,
  -7.9591e-09,  3.9750e-08],
 ...,
 [ 5.6013e-08,  7.5526e-08,  4.4496e-08,  ..., -4.4128e-08,
```

```
   -5.9930e-08, -1.8247e-08],
  [ 7.7614e-08,  9.8348e-08,  1.0455e-07,  ...,  6.3272e-08,
    4.1781e-08,  4.5901e-08],
  [ 5.9834e-08,  7.1006e-08,  9.0437e-08,  ...,  1.1654e-07,
    8.7550e-08,  9.8837e-08]],

 [[-4.3810e-08,  1.3270e-08,  7.8275e-09,  ..., -5.8804e-09,
   -2.6217e-08, -1.5649e-08],
  [ 4.1700e-08,  1.0778e-07,  1.0946e-07,  ...,  7.6403e-08,
    7.1450e-08,  9.7615e-08],
  [ 1.0436e-07,  1.6586e-07,  1.5933e-07,  ...,  1.3517e-07,
    1.3487e-07,  1.6449e-07],
  ...,
  [ 9.8763e-08,  1.5072e-07,  1.2547e-07,  ...,  6.8316e-08,
    6.8382e-08,  1.1367e-07],
  [ 9.1435e-08,  1.3576e-07,  1.3793e-07,  ...,  1.1678e-07,
    1.1723e-07,  1.4394e-07],
  [ 6.2183e-08,  8.8184e-08,  1.0456e-07,  ...,  1.3941e-07,
    1.3333e-07,  1.5844e-07]]],


...,


[[[-6.1896e-02, -3.0206e-02,  1.9225e-02,  ...,  4.3665e-02,
   -2.2114e-02, -4.2214e-02],
  [-3.8061e-02,  6.0774e-03,  4.5797e-02,  ...,  9.6029e-02,
    5.9254e-02,  2.9958e-02],
  [-2.9672e-02,  2.7766e-03,  2.0457e-02,  ...,  5.9828e-02,
    4.1422e-02,  2.3134e-02],
  ...,
  [ 1.1916e-02,  4.5701e-02,  4.4892e-02,  ...,  4.7419e-02,
    2.2274e-02, -5.4993e-03],
  [-3.2468e-02, -1.2210e-02,  2.2023e-02,  ...,  5.8061e-02,
   -7.5033e-03, -5.9736e-02],
  [-4.3314e-02, -2.8162e-02, -5.9126e-03,  ...,  8.8460e-02,
    8.4406e-03, -5.0019e-02]],

 [[-6.1292e-02, -1.4004e-02,  1.7229e-02,  ...,  1.8349e-02,
   -3.2708e-02, -4.1060e-02],
  [-3.1506e-02,  2.4460e-02,  4.5516e-02,  ...,  6.6806e-02,
    4.6687e-02,  3.3248e-02],
  [-3.2216e-02,  2.0718e-02,  2.3343e-02,  ...,  3.5265e-02,
    3.6478e-02,  3.1291e-02],
  ...,
  [ 1.7739e-02,  6.1040e-02,  4.8247e-02,  ...,  3.7785e-02,
    2.8894e-02,  1.3984e-02],
  [-1.0890e-02,  2.2079e-02,  4.2737e-02,  ...,  6.0247e-02,
    1.6197e-02, -1.2493e-02],
  [-2.2284e-02,  1.3220e-02,  3.0897e-02,  ...,  1.0403e-01,
    4.0119e-02, -5.3310e-03]],

 [[-8.5322e-02, -4.2603e-02,  6.8145e-03,  ...,  3.0751e-02,
   -3.4818e-02, -4.9945e-02],
  [-2.9215e-02,  1.8165e-02,  5.1092e-02,  ...,  9.0200e-02,
    5.3438e-02,  4.0169e-02],
```

```
        [-3.9932e-02, -1.1100e-03,  9.6176e-03,  ...,  2.4114e-02,
          2.6298e-02,  2.5489e-02],
        ...,
        [-3.1890e-03,  3.0454e-02,  1.6316e-02,  ...,  5.5054e-03,
         -6.2689e-03, -8.4638e-03],
        [-2.2995e-02, -2.8211e-03,  2.3203e-02,  ...,  3.5888e-02,
         -1.4296e-02, -3.2419e-02],
        [-9.8894e-03,  7.0542e-03,  1.0659e-02,  ...,  7.0495e-02,
          1.2996e-02, -8.3417e-03]]],


      [[[-7.8699e-03,  1.9911e-02,  3.4208e-02,  ...,  2.8694e-02,
          1.2820e-02,  1.8142e-02],
        [ 8.7942e-03, -3.2875e-02, -3.5713e-02,  ...,  7.2533e-02,
          4.5889e-02,  5.2383e-02],
        [-3.6122e-02, -1.1878e-01, -1.3767e-01,  ...,  3.3811e-02,
          3.7806e-02,  2.6944e-02],
        ...,
        [ 1.7322e-02,  3.9589e-03, -8.2269e-03,  ...,  2.7543e-03,
          1.8313e-02,  1.6057e-02],
        [-9.5007e-04,  1.6428e-02,  1.7156e-02,  ...,  3.3672e-03,
          2.2857e-02,  6.5783e-04],
        [ 6.1727e-03,  2.7145e-02,  1.4340e-02,  ...,  7.5867e-03,
          1.8770e-02,  1.5624e-02]],

       [[-1.3423e-02, -5.0696e-04,  8.0959e-03,  ..., -6.0963e-03,
          9.2341e-03,  1.5751e-02],
        [-1.8343e-02, -6.7982e-02, -7.0685e-02,  ...,  2.9855e-02,
          2.6264e-02,  2.3773e-02],
        [-5.4359e-02, -1.4663e-01, -1.6211e-01,  ...,  1.1781e-02,
          3.2477e-02,  1.1980e-02],
        ...,
        [ 8.3686e-04, -1.7564e-02, -1.9535e-02,  ..., -4.1382e-03,
          2.4658e-02,  1.2893e-02],
        [-6.3183e-04,  1.1788e-02,  2.4810e-02,  ...,  6.1105e-03,
          3.9210e-02,  9.6696e-03],
        [-7.1831e-03,  6.6918e-03,  5.2723e-03,  ..., -7.6077e-03,
          2.7253e-02,  1.7735e-02]],

       [[-2.3753e-04, -4.9343e-03,  2.2991e-03,  ..., -4.7958e-02,
         -2.6154e-02, -2.3525e-02],
        [-3.3053e-04, -5.1502e-02, -5.9977e-02,  ..., -1.7369e-02,
         -2.3337e-02, -3.7312e-02],
        [-2.2674e-02, -9.9412e-02, -1.1176e-01,  ..., -1.1725e-02,
         -8.3744e-03, -4.0615e-02],
        ...,
        [ 1.1437e-02, -8.0313e-03, -1.4955e-03,  ..., -3.4133e-02,
         -8.7267e-03, -2.3526e-02],
        [ 2.9522e-03,  6.7770e-04,  1.9933e-02,  ..., -2.2002e-02,
          1.4814e-02, -1.4487e-02],
        [-1.9085e-02, -2.9430e-02, -2.3284e-02,  ..., -4.8587e-02,
         -1.3049e-02, -2.4368e-02]]],


      [[[-3.6296e-02,  7.1996e-03,  1.9100e-02,  ...,  1.9602e-02,
          1.4870e-02, -1.7298e-02],
```

```
           [-1.1061e-02,  8.5665e-02,  1.2667e-01,  ...,  1.3744e-02,
            -5.5036e-05, -3.0162e-02],
           [ 1.1322e-01,  1.8634e-01,  5.0658e-02,  ..., -1.7333e-01,
            -7.2041e-02, -6.2474e-02],
           ...,
           [-5.3062e-02, -2.5781e-01, -2.6747e-01,  ...,  2.6781e-01,
             1.4344e-01,  5.5145e-02],
           [-2.1009e-02, -2.9969e-02,  1.0245e-01,  ...,  2.0843e-01,
            -4.1518e-03, -3.8118e-02],
           [-2.2155e-02,  1.2380e-02,  8.4302e-02,  ..., -4.4992e-02,
            -1.4687e-01, -9.0890e-02]],

          [[-5.3969e-03,  3.2799e-02,  1.5486e-02,  ..., -7.7451e-03,
             3.0229e-03,  1.1216e-03],
           [ 6.1723e-02,  1.4899e-01,  1.4645e-01,  ..., -2.8897e-02,
            -2.0227e-02, -9.1878e-03],
           [ 1.6146e-01,  2.0886e-01, -2.5589e-02,  ..., -2.7278e-01,
            -1.0735e-01, -6.2971e-02],
           ...,
           [-1.3723e-01, -4.0863e-01, -3.8551e-01,  ...,  4.0846e-01,
             2.6202e-01,  1.3491e-01],
           [-5.9388e-02, -6.1187e-02,  1.4197e-01,  ...,  3.5780e-01,
             9.0893e-02, -1.7392e-03],
           [ 7.8613e-03,  5.8403e-02,  1.5339e-01,  ...,  4.7045e-02,
            -1.0095e-01, -9.7920e-02]],

          [[-5.6799e-03,  1.3425e-02, -2.6461e-02,  ...,  4.4881e-03,
             2.0666e-03,  1.3902e-02],
           [ 6.5943e-03,  4.5181e-02,  6.0260e-02,  ...,  1.4368e-02,
            -5.0725e-03,  4.0505e-03],
           [ 5.5257e-02,  1.2397e-01,  4.3193e-02,  ..., -1.4486e-01,
            -7.4489e-02, -5.7533e-02],
           ...,
           [-3.1513e-02, -1.6334e-01, -1.5795e-01,  ...,  2.2904e-01,
             1.2017e-01,  7.1998e-02],
           [-1.0456e-02, -1.1248e-03,  8.4582e-02,  ...,  1.5748e-01,
             2.2142e-02, -1.0083e-02],
           [-4.8639e-03, -5.0065e-03,  3.6341e-02,  ..., -2.4361e-02,
            -7.1195e-02, -6.6788e-02]]]], requires_grad=True)
++++++++++++++++++++++++++++++++++++
model.fc:
 Linear(in_features=512, out_features=1000, bias=True)
model.fc.weight:
 Parameter containing:
tensor([[-0.0185, -0.0705, -0.0518,  ..., -0.0390,  0.1735, -0.0410],
        [-0.0818, -0.0944,  0.0174,  ...,  0.2028, -0.0248,  0.0372],
        [-0.0332, -0.0566, -0.0242,  ..., -0.0344, -0.0227,  0.0197],
        ...,
        [-0.0103,  0.0033, -0.0359,  ..., -0.0279, -0.0115,  0.0128],
        [-0.0359, -0.0353, -0.0296,  ..., -0.0330, -0.0110, -0.0513],
        [ 0.0021, -0.0248, -0.0829,  ...,  0.0417, -0.0500,  0.0663]],
       requires_grad=True)
+++++++++++++++++ After requires_grad = False ++++++++++++++++++++
model.conv1:
 Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
model.conv1.weight:
```

```
 Parameter containing:
tensor([[[[-1.0419e-02, -6.1356e-03, -1.8098e-03,  ...,  5.6615e-02,
            1.7083e-02, -1.2694e-02],
          [ 1.1083e-02,  9.5276e-03, -1.0993e-01,  ..., -2.7124e-01,
           -1.2907e-01,  3.7424e-03],
          [-6.9434e-03,  5.9089e-02,  2.9548e-01,  ...,  5.1972e-01,
            2.5632e-01,  6.3573e-02],
          ...,
          [-2.7535e-02,  1.6045e-02,  7.2595e-02,  ..., -3.3285e-01,
           -4.2058e-01, -2.5781e-01],
          [ 3.0613e-02,  4.0960e-02,  6.2850e-02,  ...,  4.1384e-01,
            3.9359e-01,  1.6606e-01],
          [-1.3736e-02, -3.6746e-03, -2.4084e-02,  ..., -1.5070e-01,
           -8.2230e-02, -5.7828e-03]],

         [[-1.1397e-02, -2.6619e-02, -3.4641e-02,  ...,  3.2521e-02,
            6.6221e-04, -2.5743e-02],
          [ 4.5687e-02,  3.3603e-02, -1.0453e-01,  ..., -3.1253e-01,
           -1.6051e-01, -1.2826e-03],
          [-8.3730e-04,  9.8420e-02,  4.0210e-01,  ...,  7.0789e-01,
            3.6887e-01,  1.2455e-01],
          ...,
          [-5.5926e-02, -5.2239e-03,  2.7081e-02,  ..., -4.6178e-01,
           -5.7080e-01, -3.6552e-01],
          [ 3.2860e-02,  5.5574e-02,  9.9670e-02,  ...,  5.4636e-01,
            4.8276e-01,  1.9867e-01],
          [ 5.3051e-03,  6.6938e-03, -1.7254e-02,  ..., -1.4822e-01,
           -7.7248e-02,  7.2183e-04]],

         [[-2.0315e-03, -9.1617e-03,  2.1209e-02,  ...,  8.9177e-02,
            3.3655e-02, -2.0102e-02],
          [ 1.5398e-02, -1.8648e-02, -1.2591e-01,  ..., -2.5342e-01,
           -1.2980e-01, -2.7975e-02],
          [ 9.8454e-03,  4.9047e-02,  2.1699e-01,  ...,  3.4872e-01,
            1.0433e-01,  1.8413e-02],
          ...,
          [-2.8356e-02,  1.8404e-02,  9.8647e-02,  ..., -1.1740e-01,
           -2.5760e-01, -1.5451e-01],
          [ 2.0766e-02, -2.6286e-03, -3.7825e-02,  ...,  2.4141e-01,
            2.4345e-01,  1.1796e-01],
          [ 7.4684e-04,  7.7677e-04, -1.0050e-02,  ..., -1.4865e-01,
           -1.1754e-01, -3.8350e-02]]],


        [[[-4.4154e-03, -4.0645e-03,  3.1589e-03,  ..., -3.7026e-02,
           -2.5158e-02, -4.7945e-02],
          [ 5.1310e-02,  5.3402e-02,  8.0436e-02,  ...,  1.4480e-01,
            1.4287e-01,  1.2312e-01],
          [-7.3337e-03,  2.1755e-03,  3.7580e-02,  ...,  6.1517e-02,
            8.0324e-02,  1.1715e-01],
          ...,
          [-2.6754e-02, -1.2297e-01, -1.3653e-01,  ..., -1.4068e-01,
           -1.1155e-01, -4.9556e-02],
          [ 2.3524e-02, -1.7288e-02, -1.1122e-02,  ..., -1.8826e-02,
           -2.3320e-02, -2.9474e-02],
          [ 2.8689e-02,  2.1659e-02,  4.7888e-02,  ...,  2.5498e-02,
```

```
    3.5346e-02,  1.1280e-02]],

  [[ 4.6919e-04,  1.2153e-02,  4.2035e-02,  ...,  4.6403e-02,
     4.0423e-02, -1.4439e-02],
   [ 4.3463e-02,  6.8779e-02,  1.3268e-01,  ...,  2.8606e-01,
     2.6905e-01,  2.0935e-01],
   [-5.7621e-02, -2.2642e-02,  3.0547e-02,  ...,  1.3763e-01,
     1.6538e-01,  1.7946e-01],
   ...,
   [-1.0816e-01, -2.5227e-01, -2.9742e-01,  ..., -2.8503e-01,
    -2.1493e-01, -1.0320e-01],
   [ 4.0709e-02, -3.2771e-02, -6.3450e-02,  ..., -9.2360e-02,
    -6.9876e-02, -4.9841e-02],
   [ 8.2942e-02,  8.7580e-02,  1.0111e-01,  ...,  5.2714e-02,
     6.0968e-02,  4.1198e-02]],

  [[-1.6391e-02, -1.3870e-02,  5.2810e-03,  ...,  4.3698e-02,
     2.2707e-02, -4.5983e-02],
   [ 3.3202e-02,  4.2014e-02,  9.3500e-02,  ...,  2.6162e-01,
     2.2970e-01,  1.6694e-01],
   [-4.5987e-02, -1.6365e-02,  2.6811e-02,  ...,  1.4951e-01,
     1.3216e-01,  1.3579e-01],
   ...,
   [-7.2129e-02, -1.8902e-01, -2.3389e-01,  ..., -1.9038e-01,
    -1.5609e-01, -7.5974e-02],
   [ 5.1161e-02, -2.5815e-02, -6.9357e-02,  ..., -5.8999e-02,
    -6.1550e-02, -4.4555e-02],
   [ 1.1174e-01,  7.8979e-02,  6.5849e-02,  ...,  3.1617e-02,
     2.5221e-02,  7.4257e-03]]],


[[[-7.0826e-08, -6.4306e-08, -7.3806e-08,  ..., -9.8000e-08,
    -1.0905e-07, -8.3421e-08],
   [-6.1125e-09,  2.0613e-09, -8.0922e-09,  ..., -4.9840e-08,
    -4.3836e-08, -3.0538e-09],
   [ 7.1953e-08,  7.5616e-08,  5.9282e-08,  ..., -9.7509e-09,
    -1.0951e-09,  4.2442e-08],
   ...,
   [ 9.5889e-08,  1.0039e-07,  7.9817e-08,  ..., -1.7491e-08,
    -4.7666e-08, -1.3265e-08],
   [ 1.2904e-07,  1.4762e-07,  1.7477e-07,  ...,  1.3233e-07,
     1.0628e-07,  9.3316e-08],
   [ 1.2558e-07,  1.3644e-07,  1.8431e-07,  ...,  2.1399e-07,
     1.7710e-07,  1.7166e-07]],

  [[-1.2690e-07, -9.6139e-08, -1.0372e-07,  ..., -1.1808e-07,
    -1.3309e-07, -1.0820e-07],
   [-5.7412e-08, -2.5055e-08, -3.0115e-08,  ..., -7.2922e-08,
    -6.7022e-08, -2.2574e-08],
   [ 2.1813e-08,  4.8608e-08,  3.1222e-08,  ..., -1.8694e-08,
    -7.9591e-09,  3.9750e-08],
   ...,
   [ 5.6013e-08,  7.5526e-08,  4.4496e-08,  ..., -4.4128e-08,
    -5.9930e-08, -1.8247e-08],
   [ 7.7614e-08,  9.8348e-08,  1.0455e-07,  ...,  6.3272e-08,
     4.1781e-08,  4.5901e-08],
```

```
  [ 5.9834e-08,  7.1006e-08,  9.0437e-08,  ...,  1.1654e-07,
    8.7550e-08,  9.8837e-08]],

 [[-4.3810e-08,  1.3270e-08,  7.8275e-09,  ..., -5.8804e-09,
   -2.6217e-08, -1.5649e-08],
  [ 4.1700e-08,  1.0778e-07,  1.0946e-07,  ...,  7.6403e-08,
    7.1450e-08,  9.7615e-08],
  [ 1.0436e-07,  1.6586e-07,  1.5933e-07,  ...,  1.3517e-07,
    1.3487e-07,  1.6449e-07],
  ...,
  [ 9.8763e-08,  1.5072e-07,  1.2547e-07,  ...,  6.8316e-08,
    6.8382e-08,  1.1367e-07],
  [ 9.1435e-08,  1.3576e-07,  1.3793e-07,  ...,  1.1678e-07,
    1.1723e-07,  1.4394e-07],
  [ 6.2183e-08,  8.8184e-08,  1.0456e-07,  ...,  1.3941e-07,
    1.3333e-07,  1.5844e-07]]],


...,


[[[-6.1896e-02, -3.0206e-02,  1.9225e-02,  ...,  4.3665e-02,
   -2.2114e-02, -4.2214e-02],
  [-3.8061e-02,  6.0774e-03,  4.5797e-02,  ...,  9.6029e-02,
    5.9254e-02,  2.9958e-02],
  [-2.9672e-02,  2.7766e-03,  2.0457e-02,  ...,  5.9828e-02,
    4.1422e-02,  2.3134e-02],
  ...,
  [ 1.1916e-02,  4.5701e-02,  4.4892e-02,  ...,  4.7419e-02,
    2.2274e-02, -5.4993e-03],
  [-3.2468e-02, -1.2210e-02,  2.2023e-02,  ...,  5.8061e-02,
   -7.5033e-03, -5.9736e-02],
  [-4.3314e-02, -2.8162e-02, -5.9126e-03,  ...,  8.8460e-02,
    8.4406e-03, -5.0019e-02]],

 [[-6.1292e-02, -1.4004e-02,  1.7229e-02,  ...,  1.8349e-02,
   -3.2708e-02, -4.1060e-02],
  [-3.1506e-02,  2.4460e-02,  4.5516e-02,  ...,  6.6806e-02,
    4.6687e-02,  3.3248e-02],
  [-3.2216e-02,  2.0718e-02,  2.3343e-02,  ...,  3.5265e-02,
    3.6478e-02,  3.1291e-02],
  ...,
  [ 1.7739e-02,  6.1040e-02,  4.8247e-02,  ...,  3.7785e-02,
    2.8894e-02,  1.3984e-02],
  [-1.0890e-02,  2.2079e-02,  4.2737e-02,  ...,  6.0247e-02,
    1.6197e-02, -1.2493e-02],
  [-2.2284e-02,  1.3220e-02,  3.0897e-02,  ...,  1.0403e-01,
    4.0119e-02, -5.3310e-03]],

 [[-8.5322e-02, -4.2603e-02,  6.8145e-03,  ...,  3.0751e-02,
   -3.4818e-02, -4.9945e-02],
  [-2.9215e-02,  1.8165e-02,  5.1092e-02,  ...,  9.0200e-02,
    5.3438e-02,  4.0169e-02],
  [-3.9932e-02, -1.1100e-03,  9.6176e-03,  ...,  2.4114e-02,
    2.6298e-02,  2.5489e-02],
  ...,
```

```
  [-3.1890e-03,  3.0454e-02,  1.6316e-02,  ...,  5.5054e-03,
   -6.2689e-03, -8.4638e-03],
  [-2.2995e-02, -2.8211e-03,  2.3203e-02,  ...,  3.5888e-02,
   -1.4296e-02, -3.2419e-02],
  [-9.8894e-03,  7.0542e-03,  1.0659e-02,  ...,  7.0495e-02,
    1.2996e-02, -8.3417e-03]]],


[[[-7.8699e-03,  1.9911e-02,  3.4208e-02,  ...,  2.8694e-02,
    1.2820e-02,  1.8142e-02],
  [ 8.7942e-03, -3.2875e-02, -3.5713e-02,  ...,  7.2533e-02,
    4.5889e-02,  5.2383e-02],
  [-3.6122e-02, -1.1878e-01, -1.3767e-01,  ...,  3.3811e-02,
    3.7806e-02,  2.6944e-02],
  ...,
  [ 1.7322e-02,  3.9589e-03, -8.2269e-03,  ...,  2.7543e-03,
    1.8313e-02,  1.6057e-02],
  [-9.5007e-04,  1.6428e-02,  1.7156e-02,  ...,  3.3672e-03,
    2.2857e-02,  6.5783e-04],
  [ 6.1727e-03,  2.7145e-02,  1.4340e-02,  ...,  7.5867e-03,
    1.8770e-02,  1.5624e-02]],

 [[-1.3423e-02, -5.0696e-04,  8.0959e-03,  ..., -6.0963e-03,
    9.2341e-03,  1.5751e-02],
  [-1.8343e-02, -6.7982e-02, -7.0685e-02,  ...,  2.9855e-02,
    2.6264e-02,  2.3773e-02],
  [-5.4359e-02, -1.4663e-01, -1.6211e-01,  ...,  1.1781e-02,
    3.2477e-02,  1.1980e-02],
  ...,
  [ 8.3686e-04, -1.7564e-02, -1.9535e-02,  ..., -4.1382e-03,
    2.4658e-02,  1.2893e-02],
  [-6.3183e-04,  1.1788e-02,  2.4810e-02,  ...,  6.1105e-03,
    3.9210e-02,  9.6696e-03],
  [-7.1831e-03,  6.6918e-03,  5.2723e-03,  ..., -7.6077e-03,
    2.7253e-02,  1.7735e-02]],

 [[-2.3753e-04, -4.9343e-03,  2.2991e-03,  ..., -4.7958e-02,
   -2.6154e-02, -2.3525e-02],
  [-3.3053e-04, -5.1502e-02, -5.9977e-02,  ..., -1.7369e-02,
   -2.3337e-02, -3.7312e-02],
  [-2.2674e-02, -9.9412e-02, -1.1176e-01,  ..., -1.1725e-02,
   -8.3744e-03, -4.0615e-02],
  ...,
  [ 1.1437e-02, -8.0313e-03, -1.4955e-03,  ..., -3.4133e-02,
   -8.7267e-03, -2.3526e-02],
  [ 2.9522e-03,  6.7770e-04,  1.9933e-02,  ..., -2.2002e-02,
    1.4814e-02, -1.4487e-02],
  [-1.9085e-02, -2.9430e-02, -2.3284e-02,  ..., -4.8587e-02,
   -1.3049e-02, -2.4368e-02]]],


[[[-3.6296e-02,  7.1996e-03,  1.9100e-02,  ...,  1.9602e-02,
    1.4870e-02, -1.7298e-02],
  [-1.1061e-02,  8.5665e-02,  1.2667e-01,  ...,  1.3744e-02,
   -5.5036e-05, -3.0162e-02],
  [ 1.1322e-01,  1.8634e-01,  5.0658e-02,  ..., -1.7333e-01,
```

```
           -7.2041e-02, -6.2474e-02],
          ...,
          [-5.3062e-02, -2.5781e-01, -2.6747e-01,  ...,  2.6781e-01,
            1.4344e-01,  5.5145e-02],
          [-2.1009e-02, -2.9969e-02,  1.0245e-01,  ...,  2.0843e-01,
           -4.1518e-03, -3.8118e-02],
          [-2.2155e-02,  1.2380e-02,  8.4302e-02,  ..., -4.4992e-02,
           -1.4687e-01, -9.0890e-02]],

         [[-5.3969e-03,  3.2799e-02,  1.5486e-02,  ..., -7.7451e-03,
            3.0229e-03,  1.1216e-03],
          [ 6.1723e-02,  1.4899e-01,  1.4645e-01,  ..., -2.8897e-02,
           -2.0227e-02, -9.1878e-03],
          [ 1.6146e-01,  2.0886e-01, -2.5589e-02,  ..., -2.7278e-01,
           -1.0735e-01, -6.2971e-02],
          ...,
          [-1.3723e-01, -4.0863e-01, -3.8551e-01,  ...,  4.0846e-01,
            2.6202e-01,  1.3491e-01],
          [-5.9388e-02, -6.1187e-02,  1.4197e-01,  ...,  3.5780e-01,
            9.0893e-02, -1.7392e-03],
          [ 7.8613e-03,  5.8403e-02,  1.5339e-01,  ...,  4.7045e-02,
           -1.0095e-01, -9.7920e-02]],

         [[-5.6799e-03,  1.3425e-02, -2.6461e-02,  ...,  4.4881e-03,
            2.0666e-03,  1.3902e-02],
          [ 6.5943e-03,  4.5181e-02,  6.0260e-02,  ...,  1.4368e-02,
           -5.0725e-03,  4.0505e-03],
          [ 5.5257e-02,  1.2397e-01,  4.3193e-02,  ..., -1.4486e-01,
           -7.4489e-02, -5.7533e-02],
          ...,
          [-3.1513e-02, -1.6334e-01, -1.5795e-01,  ...,  2.2904e-01,
            1.2017e-01,  7.1998e-02],
          [-1.0456e-02, -1.1248e-03,  8.4582e-02,  ...,  1.5748e-01,
            2.2142e-02, -1.0083e-02],
          [-4.8639e-03, -5.0065e-03,  3.6341e-02,  ..., -2.4361e-02,
           -7.1195e-02, -6.6788e-02]]])
++++++++++++++++++++++++++++++++++++
model.fc:
 Linear(in_features=512, out_features=1000, bias=True)
model.fc.weight:
 Parameter containing:
tensor([[-0.0185, -0.0705, -0.0518,  ..., -0.0390,  0.1735, -0.0410],
        [-0.0818, -0.0944,  0.0174,  ...,  0.2028, -0.0248,  0.0372],
        [-0.0332, -0.0566, -0.0242,  ..., -0.0344, -0.0227,  0.0197],
        ...,
        [-0.0103,  0.0033, -0.0359,  ..., -0.0279, -0.0115,  0.0128],
        [-0.0359, -0.0353, -0.0296,  ..., -0.0330, -0.0110, -0.0513],
        [ 0.0021, -0.0248, -0.0829,  ...,  0.0417, -0.0500,  0.0663]])
```

Notice that the `requires_grad=True` disapears after setting `param.requires_grad = False`

Let's say we want to finetune the model on a new dataset with 10 labels. In resnet, the classifier is the last linear layer `model.fc`. We can simply replace it with a new linear layer (unfrozen by default) that acts as our classifier.

In [21]:
```
model.fc = nn.Linear(512, 10)
```

Now all parameters in the model, except the parameters of `model.fc`, are frozen. The only parameters that compute gradients are the weights and bias of `model.fc`. This means also that during traininig, the only parameters that are computing gradients (and hence updated in gradient descent) are the weights and bias of the classifier (`model.fc`).

NOTE:

The same exclusionary functionality is available as a context manager in [`torch.no_grad()`] (https://pytorch.org/docs/stable/generated/torch.no_grad.html), that we will be using for *Transfer Learning*

In [22]:
```
print("model.conv1:\n", model.conv1 )
print("model.conv1.weight:\n", model.conv1.weight)
print("++++++++++++++++++++++++++++++++++++")
print("model.fc:\n", model.fc )
print("model.fc.weight:\n", model.fc.weight)
```

```
model.conv1:
 Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
model.conv1.weight:
 Parameter containing:
tensor([[[[-1.0419e-02, -6.1356e-03, -1.8098e-03,  ...,  5.6615e-02,
            1.7083e-02, -1.2694e-02],
          [ 1.1083e-02,  9.5276e-03, -1.0993e-01,  ..., -2.7124e-01,
           -1.2907e-01,  3.7424e-03],
          [-6.9434e-03,  5.9089e-02,  2.9548e-01,  ...,  5.1972e-01,
            2.5632e-01,  6.3573e-02],
          ...,
          [-2.7535e-02,  1.6045e-02,  7.2595e-02,  ..., -3.3285e-01,
           -4.2058e-01, -2.5781e-01],
          [ 3.0613e-02,  4.0960e-02,  6.2850e-02,  ...,  4.1384e-01,
            3.9359e-01,  1.6606e-01],
          [-1.3736e-02, -3.6746e-03, -2.4084e-02,  ..., -1.5070e-01,
           -8.2230e-02, -5.7828e-03]],

         [[-1.1397e-02, -2.6619e-02, -3.4641e-02,  ...,  3.2521e-02,
            6.6221e-04, -2.5743e-02],
          [ 4.5687e-02,  3.3603e-02, -1.0453e-01,  ..., -3.1253e-01,
           -1.6051e-01, -1.2826e-03],
          [-8.3730e-04,  9.8420e-02,  4.0210e-01,  ...,  7.0789e-01,
            3.6887e-01,  1.2455e-01],
          ...,
          [-5.5926e-02, -5.2239e-03,  2.7081e-02,  ..., -4.6178e-01,
           -5.7080e-01, -3.6552e-01],
          [ 3.2860e-02,  5.5574e-02,  9.9670e-02,  ...,  5.4636e-01,
            4.8276e-01,  1.9867e-01],
          [ 5.3051e-03,  6.6938e-03, -1.7254e-02,  ..., -1.4822e-01,
           -7.7248e-02,  7.2183e-04]],

         [[-2.0315e-03, -9.1617e-03,  2.1209e-02,  ...,  8.9177e-02,
            3.3655e-02, -2.0102e-02],
          [ 1.5398e-02, -1.8648e-02, -1.2591e-01,  ..., -2.5342e-01,
           -1.2980e-01, -2.7975e-02],
          [ 9.8454e-03,  4.9047e-02,  2.1699e-01,  ...,  3.4872e-01,
            1.0433e-01,  1.8413e-02],
          ...,
          [-2.8356e-02,  1.8404e-02,  9.8647e-02,  ..., -1.1740e-01,
           -2.5760e-01, -1.5451e-01],
          [ 2.0766e-02, -2.6286e-03, -3.7825e-02,  ...,  2.4141e-01,
            2.4345e-01,  1.1796e-01],
          [ 7.4684e-04,  7.7677e-04, -1.0050e-02,  ..., -1.4865e-01,
           -1.1754e-01, -3.8350e-02]]],


        [[[-4.4154e-03, -4.0645e-03,  3.1589e-03,  ..., -3.7026e-02,
           -2.5158e-02, -4.7945e-02],
          [ 5.1310e-02,  5.3402e-02,  8.0436e-02,  ...,  1.4480e-01,
            1.4287e-01,  1.2312e-01],
          [-7.3337e-03,  2.1755e-03,  3.7580e-02,  ...,  6.1517e-02,
            8.0324e-02,  1.1715e-01],
          ...,
          [-2.6754e-02, -1.2297e-01, -1.3653e-01,  ..., -1.4068e-01,
           -1.1155e-01, -4.9556e-02],
```

```
 [ 2.3524e-02, -1.7288e-02, -1.1122e-02,  ..., -1.8826e-02,
  -2.3320e-02, -2.9474e-02],
 [ 2.8689e-02,  2.1659e-02,  4.7888e-02,  ...,  2.5498e-02,
   3.5346e-02,  1.1280e-02]],

[[ 4.6919e-04,  1.2153e-02,  4.2035e-02,  ...,  4.6403e-02,
   4.0423e-02, -1.4439e-02],
 [ 4.3463e-02,  6.8779e-02,  1.3268e-01,  ...,  2.8606e-01,
   2.6905e-01,  2.0935e-01],
 [-5.7621e-02, -2.2642e-02,  3.0547e-02,  ...,  1.3763e-01,
   1.6538e-01,  1.7946e-01],
 ...,
 [-1.0816e-01, -2.5227e-01, -2.9742e-01,  ..., -2.8503e-01,
  -2.1493e-01, -1.0320e-01],
 [ 4.0709e-02, -3.2771e-02, -6.3450e-02,  ..., -9.2360e-02,
  -6.9876e-02, -4.9841e-02],
 [ 8.2942e-02,  8.7580e-02,  1.0111e-01,  ...,  5.2714e-02,
   6.0968e-02,  4.1198e-02]],

[[-1.6391e-02, -1.3870e-02,  5.2810e-03,  ...,  4.3698e-02,
   2.2707e-02, -4.5983e-02],
 [ 3.3202e-02,  4.2014e-02,  9.3500e-02,  ...,  2.6162e-01,
   2.2970e-01,  1.6694e-01],
 [-4.5987e-02, -1.6365e-02,  2.6811e-02,  ...,  1.4951e-01,
   1.3216e-01,  1.3579e-01],
 ...,
 [-7.2129e-02, -1.8902e-01, -2.3389e-01,  ..., -1.9038e-01,
  -1.5609e-01, -7.5974e-02],
 [ 5.1161e-02, -2.5815e-02, -6.9357e-02,  ..., -5.8999e-02,
  -6.1550e-02, -4.4555e-02],
 [ 1.1174e-01,  7.8979e-02,  6.5849e-02,  ...,  3.1617e-02,
   2.5221e-02,  7.4257e-03]]],


[[[-7.0826e-08, -6.4306e-08, -7.3806e-08,  ..., -9.8000e-08,
  -1.0905e-07, -8.3421e-08],
 [-6.1125e-09,  2.0613e-09, -8.0922e-09,  ..., -4.9840e-08,
  -4.3836e-08, -3.0538e-09],
 [ 7.1953e-08,  7.5616e-08,  5.9282e-08,  ..., -9.7509e-09,
  -1.0951e-09,  4.2442e-08],
 ...,
 [ 9.5889e-08,  1.0039e-07,  7.9817e-08,  ..., -1.7491e-08,
  -4.7666e-08, -1.3265e-08],
 [ 1.2904e-07,  1.4762e-07,  1.7477e-07,  ...,  1.3233e-07,
   1.0628e-07,  9.3316e-08],
 [ 1.2558e-07,  1.3644e-07,  1.8431e-07,  ...,  2.1399e-07,
   1.7710e-07,  1.7166e-07]],

[[-1.2690e-07, -9.6139e-08, -1.0372e-07,  ..., -1.1808e-07,
  -1.3309e-07, -1.0820e-07],
 [-5.7412e-08, -2.5055e-08, -3.0115e-08,  ..., -7.2922e-08,
  -6.7022e-08, -2.2574e-08],
 [ 2.1813e-08,  4.8608e-08,  3.1222e-08,  ..., -1.8694e-08,
  -7.9591e-09,  3.9750e-08],
 ...,
 [ 5.6013e-08,  7.5526e-08,  4.4496e-08,  ..., -4.4128e-08,
```

```
   -5.9930e-08, -1.8247e-08],
  [ 7.7614e-08,  9.8348e-08,  1.0455e-07,  ...,  6.3272e-08,
    4.1781e-08,  4.5901e-08],
  [ 5.9834e-08,  7.1006e-08,  9.0437e-08,  ...,  1.1654e-07,
    8.7550e-08,  9.8837e-08]],

 [[-4.3810e-08,  1.3270e-08,  7.8275e-09,  ..., -5.8804e-09,
   -2.6217e-08, -1.5649e-08],
  [ 4.1700e-08,  1.0778e-07,  1.0946e-07,  ...,  7.6403e-08,
    7.1450e-08,  9.7615e-08],
  [ 1.0436e-07,  1.6586e-07,  1.5933e-07,  ...,  1.3517e-07,
    1.3487e-07,  1.6449e-07],
  ...,
  [ 9.8763e-08,  1.5072e-07,  1.2547e-07,  ...,  6.8316e-08,
    6.8382e-08,  1.1367e-07],
  [ 9.1435e-08,  1.3576e-07,  1.3793e-07,  ...,  1.1678e-07,
    1.1723e-07,  1.4394e-07],
  [ 6.2183e-08,  8.8184e-08,  1.0456e-07,  ...,  1.3941e-07,
    1.3333e-07,  1.5844e-07]]],


...,


[[[-6.1896e-02, -3.0206e-02,  1.9225e-02,  ...,  4.3665e-02,
   -2.2114e-02, -4.2214e-02],
  [-3.8061e-02,  6.0774e-03,  4.5797e-02,  ...,  9.6029e-02,
    5.9254e-02,  2.9958e-02],
  [-2.9672e-02,  2.7766e-03,  2.0457e-02,  ...,  5.9828e-02,
    4.1422e-02,  2.3134e-02],
  ...,
  [ 1.1916e-02,  4.5701e-02,  4.4892e-02,  ...,  4.7419e-02,
    2.2274e-02, -5.4993e-03],
  [-3.2468e-02, -1.2210e-02,  2.2023e-02,  ...,  5.8061e-02,
   -7.5033e-03, -5.9736e-02],
  [-4.3314e-02, -2.8162e-02, -5.9126e-03,  ...,  8.8460e-02,
    8.4406e-03, -5.0019e-02]],

 [[-6.1292e-02, -1.4004e-02,  1.7229e-02,  ...,  1.8349e-02,
   -3.2708e-02, -4.1060e-02],
  [-3.1506e-02,  2.4460e-02,  4.5516e-02,  ...,  6.6806e-02,
    4.6687e-02,  3.3248e-02],
  [-3.2216e-02,  2.0718e-02,  2.3343e-02,  ...,  3.5265e-02,
    3.6478e-02,  3.1291e-02],
  ...,
  [ 1.7739e-02,  6.1040e-02,  4.8247e-02,  ...,  3.7785e-02,
    2.8894e-02,  1.3984e-02],
  [-1.0890e-02,  2.2079e-02,  4.2737e-02,  ...,  6.0247e-02,
    1.6197e-02, -1.2493e-02],
  [-2.2284e-02,  1.3220e-02,  3.0897e-02,  ...,  1.0403e-01,
    4.0119e-02, -5.3310e-03]],

 [[-8.5322e-02, -4.2603e-02,  6.8145e-03,  ...,  3.0751e-02,
   -3.4818e-02, -4.9945e-02],
  [-2.9215e-02,  1.8165e-02,  5.1092e-02,  ...,  9.0200e-02,
    5.3438e-02,  4.0169e-02],
```

```
        [-3.9932e-02, -1.1100e-03,  9.6176e-03,  ...,  2.4114e-02,
          2.6298e-02,  2.5489e-02],
        ...,
        [-3.1890e-03,  3.0454e-02,  1.6316e-02,  ...,  5.5054e-03,
         -6.2689e-03, -8.4638e-03],
        [-2.2995e-02, -2.8211e-03,  2.3203e-02,  ...,  3.5888e-02,
         -1.4296e-02, -3.2419e-02],
        [-9.8894e-03,  7.0542e-03,  1.0659e-02,  ...,  7.0495e-02,
          1.2996e-02, -8.3417e-03]]],


      [[[-7.8699e-03,  1.9911e-02,  3.4208e-02,  ...,  2.8694e-02,
          1.2820e-02,  1.8142e-02],
        [ 8.7942e-03, -3.2875e-02, -3.5713e-02,  ...,  7.2533e-02,
          4.5889e-02,  5.2383e-02],
        [-3.6122e-02, -1.1878e-01, -1.3767e-01,  ...,  3.3811e-02,
          3.7806e-02,  2.6944e-02],
        ...,
        [ 1.7322e-02,  3.9589e-03, -8.2269e-03,  ...,  2.7543e-03,
          1.8313e-02,  1.6057e-02],
        [-9.5007e-04,  1.6428e-02,  1.7156e-02,  ...,  3.3672e-03,
          2.2857e-02,  6.5783e-04],
        [ 6.1727e-03,  2.7145e-02,  1.4340e-02,  ...,  7.5867e-03,
          1.8770e-02,  1.5624e-02]],

       [[-1.3423e-02, -5.0696e-04,  8.0959e-03,  ..., -6.0963e-03,
          9.2341e-03,  1.5751e-02],
        [-1.8343e-02, -6.7982e-02, -7.0685e-02,  ...,  2.9855e-02,
          2.6264e-02,  2.3773e-02],
        [-5.4359e-02, -1.4663e-01, -1.6211e-01,  ...,  1.1781e-02,
          3.2477e-02,  1.1980e-02],
        ...,
        [ 8.3686e-04, -1.7564e-02, -1.9535e-02,  ..., -4.1382e-03,
          2.4658e-02,  1.2893e-02],
        [-6.3183e-04,  1.1788e-02,  2.4810e-02,  ...,  6.1105e-03,
          3.9210e-02,  9.6696e-03],
        [-7.1831e-03,  6.6918e-03,  5.2723e-03,  ..., -7.6077e-03,
          2.7253e-02,  1.7735e-02]],

       [[-2.3753e-04, -4.9343e-03,  2.2991e-03,  ..., -4.7958e-02,
         -2.6154e-02, -2.3525e-02],
        [-3.3053e-04, -5.1502e-02, -5.9977e-02,  ..., -1.7369e-02,
         -2.3337e-02, -3.7312e-02],
        [-2.2674e-02, -9.9412e-02, -1.1176e-01,  ..., -1.1725e-02,
         -8.3744e-03, -4.0615e-02],
        ...,
        [ 1.1437e-02, -8.0313e-03, -1.4955e-03,  ..., -3.4133e-02,
         -8.7267e-03, -2.3526e-02],
        [ 2.9522e-03,  6.7770e-04,  1.9933e-02,  ..., -2.2002e-02,
          1.4814e-02, -1.4487e-02],
        [-1.9085e-02, -2.9430e-02, -2.3284e-02,  ..., -4.8587e-02,
         -1.3049e-02, -2.4368e-02]]],


      [[[-3.6296e-02,  7.1996e-03,  1.9100e-02,  ...,  1.9602e-02,
          1.4870e-02, -1.7298e-02],
```

```
           [-1.1061e-02,  8.5665e-02,  1.2667e-01,  ...,  1.3744e-02,
            -5.5036e-05, -3.0162e-02],
           [ 1.1322e-01,  1.8634e-01,  5.0658e-02,  ..., -1.7333e-01,
            -7.2041e-02, -6.2474e-02],
           ...,
           [-5.3062e-02, -2.5781e-01, -2.6747e-01,  ...,  2.6781e-01,
             1.4344e-01,  5.5145e-02],
           [-2.1009e-02, -2.9969e-02,  1.0245e-01,  ...,  2.0843e-01,
            -4.1518e-03, -3.8118e-02],
           [-2.2155e-02,  1.2380e-02,  8.4302e-02,  ..., -4.4992e-02,
            -1.4687e-01, -9.0890e-02]],

          [[-5.3969e-03,  3.2799e-02,  1.5486e-02,  ..., -7.7451e-03,
             3.0229e-03,  1.1216e-03],
           [ 6.1723e-02,  1.4899e-01,  1.4645e-01,  ..., -2.8897e-02,
            -2.0227e-02, -9.1878e-03],
           [ 1.6146e-01,  2.0886e-01, -2.5589e-02,  ..., -2.7278e-01,
            -1.0735e-01, -6.2971e-02],
           ...,
           [-1.3723e-01, -4.0863e-01, -3.8551e-01,  ...,  4.0846e-01,
             2.6202e-01,  1.3491e-01],
           [-5.9388e-02, -6.1187e-02,  1.4197e-01,  ...,  3.5780e-01,
             9.0893e-02, -1.7392e-03],
           [ 7.8613e-03,  5.8403e-02,  1.5339e-01,  ...,  4.7045e-02,
            -1.0095e-01, -9.7920e-02]],

          [[-5.6799e-03,  1.3425e-02, -2.6461e-02,  ...,  4.4881e-03,
             2.0666e-03,  1.3902e-02],
           [ 6.5943e-03,  4.5181e-02,  6.0260e-02,  ...,  1.4368e-02,
            -5.0725e-03,  4.0505e-03],
           [ 5.5257e-02,  1.2397e-01,  4.3193e-02,  ..., -1.4486e-01,
            -7.4489e-02, -5.7533e-02],
           ...,
           [-3.1513e-02, -1.6334e-01, -1.5795e-01,  ...,  2.2904e-01,
             1.2017e-01,  7.1998e-02],
           [-1.0456e-02, -1.1248e-03,  8.4582e-02,  ...,  1.5748e-01,
             2.2142e-02, -1.0083e-02],
           [-4.8639e-03, -5.0065e-03,  3.6341e-02,  ..., -2.4361e-02,
            -7.1195e-02, -6.6788e-02]]])
++++++++++++++++++++++++++++++++++++
model.fc:
 Linear(in_features=512, out_features=10, bias=True)
model.fc.weight:
 Parameter containing:
tensor([[ 0.0313,  0.0096,  0.0258,  ...,  0.0060,  0.0032, -0.0439],
        [-0.0108,  0.0062, -0.0084,  ...,  0.0214, -0.0438, -0.0356],
        [ 0.0103, -0.0243, -0.0273,  ...,  0.0307, -0.0017,  0.0052],
        ...,
        [ 0.0374,  0.0011, -0.0352,  ...,  0.0320, -0.0220, -0.0333],
        [-0.0243,  0.0378,  0.0123,  ...,  0.0434, -0.0030, -0.0438],
        [-0.0207,  0.0123, -0.0436,  ..., -0.0331, -0.0007,  0.0013]],
       requires_grad=True)
```

In [ ]: